Towards a Quantum-Safe, Risk-Adaptive, and Stateful Zero Trust Architecture: A Hybrid Machine Learning Approach

Abhilash Panigrahi, Ezhil Arasi V
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
Email: ezhilarasi.v@vit.ac.in

Abstract

The paradigm of network security has decisively shifted from perimeter-based defense toward Zero Trust Architecture (ZTA). Concurrently, the imminent threat of Cryptographically Relevant Quantum Computers (CRQCs) jeopardizes the foundational public-key infrastructure underpinning current authentication mechanisms. Existing Zero Trust implementations primarily rely on stateless access control mechanisms and discrete risk evaluations, exposing them to session hijacking, state-tampering, and gradual trust degradation vulnerabilities. To address these critical vulnerabilities, this paper proposes the Quantum-Safe Risk-Adaptive Access Control (QSRAC) system. QSRAC integrates a hybrid machine learning risk engine combining Light Gradient Boosting Machine (LightGBM) and Isolation Forest with logit-calibrated risk scoring, yielding robust detection of both known and zero-day threat vectors. Furthermore, we introduce a continuous mathematical trust decay model that dynamically attenuates session trust based on contextual risk trends, coupled with a quantum-safe stateful cryptographic hash-chain mechanism. By leveraging Post-Quantum Cryptography (PQC) standards—specifically ML-KEM-512 (Kyber512) for Key Encapsulation and ML-DSA-44 (Dilithium2) for Digital Signatures—QSRAC guarantees session integrity against harvest-now-decrypt-later attacks. Extensive empirical evaluation using the UNSW-NB15 dataset demonstrates that our hybrid model achieves an F1-score of 0.9071, a Precision-Recall Area Under Curve (PR-AUC) of 0.9438, and an ROC-AUC of 0.9173. Despite the cryptographic overhead of lattice-based schemes, the system maintains a highly efficient mean inference latency of 7.91 ms by offloading continuous validation to symmetric HMAC-SHA256 chains. Furthermore, comprehensive security analysis confirms absolute resilience against replay and hash-chain tampering attacks. This paper provides a deep theoretical and mathematical exposition of the calibration mechanisms required for highly skewed security telemetry, establishing QSRAC as a scalable, state-of-the-art framework for next-generation, quantum-resilient enterprise Zero Trust environments.

Index Terms

Post-Quantum Cryptography, Zero Trust Architecture, Risk-Adaptive Access Control, Machine Learning, Intrusion Detection, Trust Management, Stateful Security, Latency Optimization, Algorithm Complexity, Cryptographic State Integrity.

I. Introduction

A. Background and Contextual Precedent

The traditional perimeter-based cybersecurity model, historically heavily reliant on rigid demarcations between "trusted" internal enterprise networks and "untrusted" external public domains, has been systematically and irreversibly dismantled. The proliferation of highly distributed cloud computing infrastructures, the ubiquity of remote workforce topologies, and the granularization of backend systems into microservice architectures have rendered traditional VPNs and firewalls insufficient. In response, the cybersecurity industry and federal standards bodies have aggressively adopted Zero Trust Architecture (ZTA), operationalizing the core architectural tenet: "never trust, always verify" [1]. A true Zero Trust ecosystem demands continuous authentication and granular authorization for every single transactional

request, irrespective of the network origin, the physical device posture, or the user's prior authentication history.

Concurrently, a secondary, systemic, and existential threat to global cybersecurity is rapidly maturing: the advent of Cryptographically Relevant Quantum Computers (CRQCs). Utilizing Shor's algorithm, a sufficiently scaled and error-corrected quantum computer will possess the capability to trivially compromise the discrete logarithm and integer factorization mathematical problems that form the bedrock of contemporary asymmetric cryptography [2]. This includes widely deployed protocols such as Rivest-Shamir-Adleman (RSA), Elliptic Curve Cryptography (ECC), and Diffie-Hellman key exchanges. The urgency of this cryptographic threat is dramatically exacerbated by the widespread deployment of "harvest-now-decrypt-later" strategies, wherein advanced persistent threats (APTs) and nation-state adversaries actively intercept, exfiltrate, and store current encrypted traffic with the explicit, long-term intent of decrypting it retrospectively once quantum capabilities are fully realized [35].

B. Motivation

While modern Risk-Adaptive Access Control (RAdAC) systems have commendably begun incorporating artificial intelligence and machine learning to evaluate behavioral context at the edge, they remain fundamentally flawed in three distinct operational and theoretical dimensions.

Firstly, the overwhelming majority of contemporary continuous authentication systems are fundamentally *stateless*. They evaluate individual API requests in an absolute vacuum, typically relying on static JSON Web Tokens (JWTs) mandated by protocols like OAuth 2.0 or OpenID Connect. While highly scalable, statelessness fails to cryptographically bind sequential requests into a cohesive, verifiable temporal chain. This architectural choice leaves user sessions highly vulnerable to sophisticated network replay attacks, token theft via cross-site scripting (XSS), and advanced session hijacking methodologies where an attacker clones a valid token and operates asynchronously.

Secondly, trust within these legacy systems is typically binary and rigidly static. Once a user successfully passes a primary authentication gateway (even with Multi-Factor Authentication), trust is unilaterally granted and persists uniformly until a hardcoded token expiration event occurs. This paradigm entirely ignores the mathematical reality that environmental risk fluctuates dynamically over time based on real-time user behavior. A user exhibiting slightly anomalous behavior over a prolonged period should naturally lose the system's trust incrementally, yet modern implementations entirely fail to map this temporal degradation through continuous mathematical formulas.

Thirdly, and most critically for the longevity of the system, existing continuous authentication frameworks rely heavily on classical cryptographic algorithms for their initial session negotiation and ongoing token signatures, rendering them entirely vulnerable to imminent quantum adversaries.

C. Problem Statement

There exists a profound, unresolved tension between continuous machine-learning risk evaluation, strict stateful session integrity, and quantum cryptographic resistance. Transitioning to Post-Quantum Cryptography (PQC) standards introduces significant computational and bandwidth overheads. Lattice-based cryptography, while mathematically resistant to quantum phase estimation attacks, requires substantially larger key sizes and signature dimensions compared to ECC. Simultaneously, continuous machine-learning-based risk inference introduces variable latency into the API path.

The central, multi-faceted problem statement of this research is therefore defined as follows: *How can we architect a highly scalable Zero Trust access control middleware that (a) continuously models high-dimensional behavioral risk and executes differential trust decay, (b) enforces strict, stateful cryptographic session integrity mathematically immune to quantum and classical threats, and (c) maintains the strict sub-10 millisecond latency profile absolutely mandated by high-throughput enterprise microservice environments?*

D. Research Gap

An extensive, critical review of the academic literature reveals that while there are significant, albeit disjointed, advancements in ML-based Intrusion Detection Systems (IDS), dynamic probabilistic trust modeling, and the theoretical standardization of PQC (e.g., NIST's formal selection of ML-KEM and ML-DSA), no existing framework successfully unifies these disparate domains into a cohesive, low-latency, and operational architecture.

Specifically, current hybrid machine learning models proposed in academic literature frequently fail to account for the highly polarized, skewed nature of real-world enterprise network traffic. This oversight inevitably leads to catastrophic "threshold collapse," wherein legitimate, slightly anomalous administrative traffic is incorrectly classified as high-risk, inducing severe operational alert fatigue. Furthermore, while theoretical mathematical trust decay models have been proposed, they critically lack concrete, low-latency cryptographic enforcement mechanisms. Without a cryptographic binding connecting the risk evaluation to the actual network state, the "continuous" aspect of continuous authentication becomes easily bypassable by malicious actors manipulating the sequential flow of HTTP requests.

E. Contributions

To bridge these critical and urgent research gaps, this paper introduces the Quantum-Safe Risk-Adaptive Access Control (QSRAC) framework. The primary technical and theoretical contributions of this research are explicitly defined as follows:

- **Quantum-Safe Stateful Security Architecture:** We design, formalize, and implement a novel cryptographic envelope hash-chain mechanism. This mechanism is secured initially by ML-KEM-512 (for quantum-safe Key Encapsulation) and ML-DSA-44 (for quantum-safe Digital Signatures), transitioning to ultra-fast symmetric HMAC-SHA256 operations to ensure strict sequence incrementation and absolute, mathematical protection against replay and state-tampering attacks.
- **Calibrated Hybrid Machine Learning Engine:** We propose a sophisticated, multi-stage hybrid ML pipeline that seamlessly integrates LightGBM (for high-speed supervised threat classification) and Isolation Forest (for unsupervised zero-day anomaly detection). Critically, we introduce a rigorous logit-based calibration transformation methodology that explicitly resolves the threshold collapse phenomenon inherently caused by polarized, highly skewed dataset distributions common in IDS telemetry.
- **Continuous Mathematical Trust Decay Model:** We formulate a dynamic, differential trust decay model governed by exponential decay equations. This engine autonomously and continuously degrades user trust based on the sensitivity of the accessed environment and localized, temporal risk trends. It actively enforces proactive step-up authentication (cryptographic MFA) before critical security thresholds are breached.
- **Comprehensive Empirical and Theoretical Validation:** We provide extensive experimental analysis utilizing the highly regarded UNSW-NB15 dataset. We demonstrate superior precision and recall (F1-score of 0.9071), prove ultra-low operational latency (mean 7.91 ms), and provide deep analytical interpretations of the resulting distribution curves, feature importance, and boundary stability.

F. Paper Organization

The remainder of this exhaustive manuscript is structured systematically to guide the reader from theoretical foundations to empirical validation. Section II presents a comprehensive, deeply analytical literature review exploring the evolution of IDS and Zero Trust. Section III establishes the formal System Design Philosophy and the limitations of modern industry implementations. Section IV details the formal Threat Model and operational security assumptions. Section V outlines the proposed QSRAC system architecture and deployment topology. Section VI provides a deep dive into Risk Distribution and Calibration Theory. Section VII establishes the rigorous mathematical formulations governing the system. Section VIII

outlines the core algorithmic implementations and their detailed computational complexities. Section IX discusses implementation constraints and atomic state management. Section X outlines the experimental setup. Section XI presents the results, followed by profound interpretations of the underlying data behavior. Section XII explores model interpretability and cost-benefit trade-offs. Section XIII documents the security analysis against simulated attack vectors. Section XIV provides practical deployment considerations and scalability constraints. Section XV offers an extended discussion of the architecture's limitations, and Section XVI concludes the paper with definitive directions for future research.

II. System Design Philosophy and Zero Trust Limitations in Industry

Before delving into the technical architecture of QSRAC, it is imperative to establish the philosophical design constraints that governed its creation, and critically analyze the limitations of contemporary industry-standard Zero Trust implementations.

A. The Friction Between Stateless REST and Stateful Security

The modern web is built upon the foundation of REpresentational State Transfer (REST), an architectural style that mandates stateless communication. In a pure REST architecture, every request from client to server must contain all the information necessary to understand and process the request, independently of any requests that preceded it. This statelessness is the primary driver of the internet's incredible horizontal scalability; a load balancer can route request N to Server A and request $N + 1$ to Server B without any synchronization overhead.

However, statelessness is fundamentally antithetical to rigorous security. Security is inherently a stateful concept. The continuous evaluation of trust requires an understanding of history: What did this user do one minute ago? Have they failed authentication three times in the past hour? Did their geographic velocity exceed physical limitations between their last request and this one?

To bridge this gap, the industry adopted JSON Web Tokens (JWTs). A JWT encapsulates the user's identity and claims into a cryptographically signed, stateless payload. The server validates the signature and trusts the claims. Yet, this approach introduces a critical vulnerability window: the Time-To-Live (TTL). If a token is stolen or intercepted, the attacker wields absolute authority until the token expires. Revocation of stateless tokens is notoriously difficult, requiring complex, centralized blocklists that re-introduce the very statefulness JWTs sought to avoid.

QSRAC Design Philosophy: QSRAC posits that security *must* be stateful, but that statefulness must not cripple horizontal scalability. We achieve this by moving the state out of the application memory and into an ultra-fast, in-memory, atomic ledger (Redis), utilizing cryptographic hash chains to ensure the client is enforcing the temporal sequence alongside the server.

B. The Fallacy of Binary Trust in Dynamic Environments

Contemporary Identity and Access Management (IAM) solutions, including leading enterprise SSO providers, frequently treat authentication as a binary threshold. A user provides a password and an MFA token; if verified, their trust state transitions from 0 to 1. This state remains at 1 until the session times out, at which point it abruptly drops to 0.

This binary step-function fails to map to human behavioral reality. A legitimate user whose device has been silently compromised by a remote access trojan (RAT) will continue to operate with a trust level of 1, even as the RAT begins enumerating network shares or exfiltrating data in the background. The system lacks the vocabulary to express "suspicion."

QSRAC Design Philosophy: Trust must be expressed as a continuous, floating-point variable $T \in [0.0, 1.0]$. Trust must be dynamically eroded by anomalous behavior and computationally restored by explicit verification actions (Step-Up MFA). The architecture must act as a damping mechanism against sudden, catastrophic breaches, offering a "gray zone" of restricted access before full termination.

C. Quantum Vulnerability in the API Gateway

The initial handshake between a client and a Zero Trust gateway typically utilizes TLS 1.3, secured by Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) for perfect forward secrecy, and RSA/ECDSA for server authentication. While mathematically robust against classical computation, Shor's algorithm executed on a CRQC will easily calculate the discrete logarithms underlying ECDHE.

The industry's hesitation to adopt PQC algorithms broadly stems from performance anxiety. For instance, an ML-KEM-512 (Kyber) public key is 800 bytes, and the ciphertext is 768 bytes, compared to roughly 32 bytes for an X25519 curve. ML-DSA-44 (Dilithium) signatures are a staggering 2,420 bytes. Injecting multi-kilobyte cryptographic payloads into every single API request in a microservice mesh would lead to catastrophic network congestion and CPU exhaustion.

QSRAC Design Philosophy: Quantum safety must be achieved without continuously paying the lattice-based cryptographic penalty. QSRAC isolates the heavy PQC operations strictly to the initial session encapsulation. Once the symmetric key ($K_{session}$) is safely established, the continuous, per-request validation is handed over to HMAC-SHA256, an algorithm that is both incredibly fast and entirely quantum-resistant (as Grover's algorithm only halves its effective security, leaving 128 bits of security, which remains practically unbreakable).

III. Literature Review

To position the QSRAC framework within the broader academic and industrial landscape, this section provides an exhaustive review of prior literature across five highly relevant sub-domains, culminating in a critical analysis of current limitations.

A. Evolution of Intrusion Detection Systems (IDS) and Machine Learning Integration

The genesis of network defense was rooted in signature-based Intrusion Detection Systems (e.g., Snort, Suricata). These systems relied on static sets of rules meticulously curated by security analysts to identify known malicious payloads [3]. While highly efficient and producing minimal false positives, signature-based systems exhibit a fatal flaw: they are entirely blind to zero-day vulnerabilities and polymorphic malware that subtly alters its signature upon transmission.

To address the brittleness of static signatures, the academic community pivoted aggressively toward Machine Learning (ML) approaches. Early integration focused on traditional algorithms. Support Vector Machines (SVM) were highly favored for their margin-maximization properties, particularly in high-dimensional packet analysis. However, SVMs scale poorly; their training complexity is $\mathcal{O}(n^2)$ to $\mathcal{O}(n^3)$, making them computationally prohibitive for datasets containing tens of millions of network flows. Naive Bayes and Logistic Regression offered faster training times but struggled to capture the complex, non-linear interactions inherent in cyber-attacks.

B. Ensemble Methods and Gradient Boosting

The computational limitations of traditional ML catalyzed the adoption of ensemble learning, specifically decision tree ensembles. Random Forest models provided excellent variance reduction by averaging multiple deep trees, yet they struggled with class imbalance—a pervasive issue in cybersecurity where anomalous traffic represents a fraction of a percent of total volume.

Gradient Boosting Decision Trees (GBDT), specifically eXtreme Gradient Boosting (XGBoost) and Light Gradient Boosting Machine (LightGBM) [4], represented a paradigm shift. GBDTs build trees sequentially, with each new tree directly targeting the residual errors of the previous ensemble. LightGBM, in particular, introduced Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). GOSS retains instances with large gradients (typically misclassified anomalies) while randomly sampling instances with small gradients (well-understood benign traffic), drastically accelerating training and inference without sacrificing precision on rare attack vectors.

However, pure supervised GBDT models suffer from concept drift. As attackers continuously modify their Tactics, Techniques, and Procedures (TTPs), the feature distribution of live traffic drifts away from the historical training data, causing supervised precision to decay over time.

C. Unsupervised Anomaly Detection Paradigms

To counteract concept drift and detect genuine zero-day exploits, researchers explored unsupervised learning. Clustering algorithms like K-Means and DBSCAN attempt to group normal behavior, labeling outliers as attacks. However, high-dimensional network data suffers from the "curse of dimensionality," where distance metrics (like Euclidean distance) lose their meaning, making clustering highly unreliable [30].

Isolation Forest [5] emerged as a superior alternative because it explicitly isolates anomalies rather than profiling normality. By randomly selecting features and split values, Isolation Forest creates isolation trees. Anomalies, being sparse and statistically distinct, require fewer splits to be isolated in a leaf node.

The primary critique of Isolation Forest in applied literature is its propensity for catastrophic false-positive rates when deployed in highly dynamic enterprise environments. Legitimate bulk-data transfers or unusual developer activities are structurally "anomalous" and are frequently flagged, causing denial-of-service to legitimate workflows.

D. Risk-Adaptive Access Control (RAdAC) and Trust Dynamics

The access control paradigm has evolved from Discretionary Access Control (DAC) to Mandatory Access Control (MAC), and subsequently to Role-Based (RBAC) and Attribute-Based Access Control (ABAC) [6], [24]. While ABAC allows for contextual decisions (e.g., granting access only during business hours), it remains largely deterministic and rule-bound.

Risk-Adaptive Access Control (RAdAC) [10] introduced the concept of dynamic, probabilistic authorization. In RAdAC, access is granted only if the operational necessity outweighs the calculated security risk. Alshehri et al. [7] advanced this by proposing continuous trust evaluation models for Zero Trust, mapping risk scores to mathematical trust metrics.

However, existing literature on trust dynamics largely focuses on the theoretical mathematical models (e.g., Markov chains, Bayesian networks) without addressing the cryptographic engineering required to enforce these models securely. A theoretical trust decay equation is useless if an attacker can manipulate the HTTP headers to spoof their identity or reset the decay timer.

E. Post-Quantum Cryptography in Distributed Networks

The standardization of PQC by NIST represents a monumental shift in global cryptographic infrastructure [8]. ML-KEM (Kyber) relies on the hardness of the Module Learning With Errors (MLWE) problem, while ML-DSA (Dilithium) is based on the Fiat-Shamir with Aborts paradigm applied to lattice structures [15], [16].

Recent literature has extensively evaluated PQC performance on resource-constrained devices, such as IoT sensors and Vehicular Ad-hoc Networks (VANETs) [9], [31]. Raza et al. [32] highlighted that while Kyber's decapsulation latency is surprisingly competitive with ECDH, the massive public key and ciphertext sizes cause severe UDP/TCP fragmentation, leading to increased packet loss and retransmission delays in noisy environments.

The literature explicitly lacks a hybrid middleware framework that completely isolates the PQC payload bloat to the initial authentication stage, transitioning seamlessly to highly efficient, quantum-safe symmetric hashing for the continuous validation lifecycle. This is the precise engineering gap filled by QSRAC.

IV. Detailed Threat Model and Attack Simulation Scenarios

To rigorously define the security guarantees of the QSRAC architecture, we must formally delineate the threat model, expanding upon the Dolev-Yao model to encompass quantum adversaries and highly motivated insider threats.

TABLE I: Comparative Summary of Access Control and Detection Frameworks

Framework / Model	Zero-Day Detection	Stateful Integrity	Quantum Safe	Trust Decay	Primary Limitation
Standard RBAC/ABAC	No	No	No	No	Rigid, binary, blind to context
Pure Supervised IDS (SVM/RF)	No	N/A	N/A	N/A	High latency, fails on novel attacks
Standalone Anomaly (IForest)	Yes	N/A	N/A	N/A	Unmanageable false positive rates
JWT-based Zero Trust	Variable	No	No	No	Vulnerable to replay and token theft
Theoretical RAdAC	Yes	No	No	Yes	Lacks cryptographic enforcement mechanisms
Proposed QSRAC	**Yes**	**Yes**	**Yes**	**Yes**	Requires Redis infrastructure overhead

A. Expanded Attacker Capabilities

The adversary, denoted as $\mathcal{A}$, possesses profound network, computational, and temporal advantages:

1) **Complete Network Dominance:** $\mathcal{A}$ commands the entire network layer between the client and the QSRAC gateway. $\mathcal{A}$ can intercept, drop, delay, reorder, and inject arbitrary TCP/IP packets at will.
2) **Quantum Computational Supremacy:** $\mathcal{A}$ operates a fully functional Cryptographically Relevant Quantum Computer (CRQC). Consequently, $\mathcal{A}$ can break RSA-2048 or ECC-256 in real-time or retrospectively. $\mathcal{A}$ can leverage Grover's algorithm to execute brute-force attacks against symmetric keys at $\mathcal{O}(\sqrt{N})$ complexity, requiring the system to utilize 256-bit symmetric keys to maintain a 128-bit post-quantum security margin.
3) **Insider Knowledge and Credential Compromise:** $\mathcal{A}$ is not solely an external actor. We assume $\mathcal{A}$ may have successfully acquired valid authentication credentials (username/password) via spear-phishing, or operates as a malicious insider attempting to escalate privileges or exfiltrate data incrementally to avoid detection.

B. Security Assumptions and Trust Anchors

A system cannot be secure without defined anchors of trust. QSRAC relies on the following assumptions:

- **Endpoint Integrity (Partial):** While $\mathcal{A}$ may compromise the user's network or credentials, we assume the physical volatile memory of the client's device is sufficiently isolated to store the symmetric $K_{session}$ key safely during the active session. If the endpoint is fully rooted (e.g., via a kernel-level keylogger and memory scraper), no remote protocol can guarantee security.
- **Cryptographic Hardness:** The NIST-standardized Module Learning With Errors (MLWE) problem underlying ML-KEM-512, and the cryptographic hash function SHA-256, remain mathematically unbreakable even against advanced quantum heuristics.
- **Backend Atomicity:** The Redis server operating behind the QSRAC gateway executes Lua scripts with absolute atomicity. A single thread evaluates the script, guaranteeing that Time-Of-Check-To-Time-Of-Use (TOCTOU) race conditions cannot be artificially induced by flooding the server with parallel requests.

C. Attack Simulation Scenarios

Based on this threat model, we define the specific attack scenarios QSRAC is designed to explicitly neutralize:

Scenario 1: The "Harvest Now, Decrypt Later" Attack. $\mathcal{A}$ intercepts the initial login handshake containing the user's credentials and the session negotiation. $\mathcal{A}$ archives this packet capture for five years until a CRQC is available. *Mitigation:* Because the session key is encapsulated using ML-KEM-512, $\mathcal{A}$ cannot decrypt the payload to extract $K_{session}$, ensuring retrospective confidentiality.

Scenario 2: Asynchronous Replay Attack. $\mathcal{A}$ intercepts a valid, highly privileged API request (e.g., `POST /api/v1/delete_database`) containing a valid Envelope Hash and Sequence Number ($Seq = 42$). $\mathcal{A}$ waits until the legitimate user logs off, and then replays the exact same packet to the server.

Mitigation: The QSRAC Redis gate compares the incoming $Seq = 42$ against the stored sequence $Seq_{stored} \geq 42$. Because $42 \not> 42$, the request is violently rejected atomically, and an alert is generated.

Scenario 3: Stateful Context Tampering (MitM). $\mathcal{A}$ intercepts a valid request where the user is transmitting from a high-risk geo-location (`geo_risk=0.9`). $\mathcal{A}$ alters the JSON payload in transit, changing `geo_risk` to 0.1 to avoid triggering the risk engine. *Mitigation:* The Envelope Hash (H_n) is mathematically bound to the exact context parameters. To produce a valid hash for the modified context, $\mathcal{A}$ must re-run the HMAC-SHA256 function. However, $\mathcal{A}$ does not possess the symmetric $K_{session}$ key. The server computes the expected hash based on the tampered context, finds a mismatch with the provided hash, and terminates the session (`CHAIN_BROKEN`).

Scenario 4: Low-and-Slow Exfiltration (Insider Threat). $\mathcal{A}$ is a malicious insider with valid credentials. To avoid triggering threshold-based IDS alarms, $\mathcal{A}$ downloads sensitive records slowly (e.g., 1 record per minute), maintaining a "Medium" risk profile that avoids immediate blocking. *Mitigation:* The continuous Trust Decay engine accumulates the Medium risk over time. While a single request is allowed, the trust score T drops from $1.0 \rightarrow 0.8 \rightarrow 0.5$. Once T breaches the confidence threshold, the Policy Engine triggers a mandatory Step-Up MFA challenge, interrupting the automated exfiltration script and demanding physical user validation.

V. System Design and Architecture

The Quantum-Safe Risk-Adaptive Access Control (QSRAC) framework is architected as a highly modular, resilient, stateful Zero Trust API gateway. It acts as an impermeable intermediary, enforcing continuous, cryptographic, and risk-aware validation of every single digital transaction before it reaches the protected microservices.

As illustrated in Fig. 1, the architecture is modularized into five deeply integrated, high-speed execution layers:

A. 1. The Quantum Cryptographic Initialization Layer

This layer exclusively handles the high-latency, computationally expensive operations required to establish a secure session, occurring strictly once per session lifecycle (e.g., upon user login).

- **Asymmetric Generation:** The client application locally generates an ephemeral ML-KEM-512 public/private keypair.
- **Authentication and Encapsulation:** The client transmits its credentials and the public Kyber key to the QSRAC gateway over standard TLS. Upon verifying credentials, the QSRAC server utilizes `liboqs` to encapsulate a highly secure, 256-bit symmetric session key ($K_{session}$) against the client's Kyber key.
- **Quantum-Safe Signatures:** To guarantee non-repudiation and prevent Man-in-the-Middle identity spoofing, the server digitally signs the initial session payload (including the core identity token) using its private ML-DSA-44 (Dilithium2) key.
- **State Initialization:** The server initializes a highly optimized state dictionary in the Redis backend, setting the initial Trust to 1.0, the Sequence to 0, and seeding the hash chain.

B. 2. The Telemetry and Context Extraction Layer

For every subsequent API request, this layer intercepts the incoming payload. It operates in strict $\mathcal{O}(1)$ time to avoid adding noticeable latency to the API path. It parses HTTP headers, IP metadata, and behavioral telemetry to extract an n-dimensional attribute vector. In this study, the context vector is defined by eight critical attributes: `hour_of_day`, `request_rate`, `failed_attempts`, `geo_risk_score`, `device_trust_score`, `sensitivity_level`, `is_vpn`, and `is_tor`.

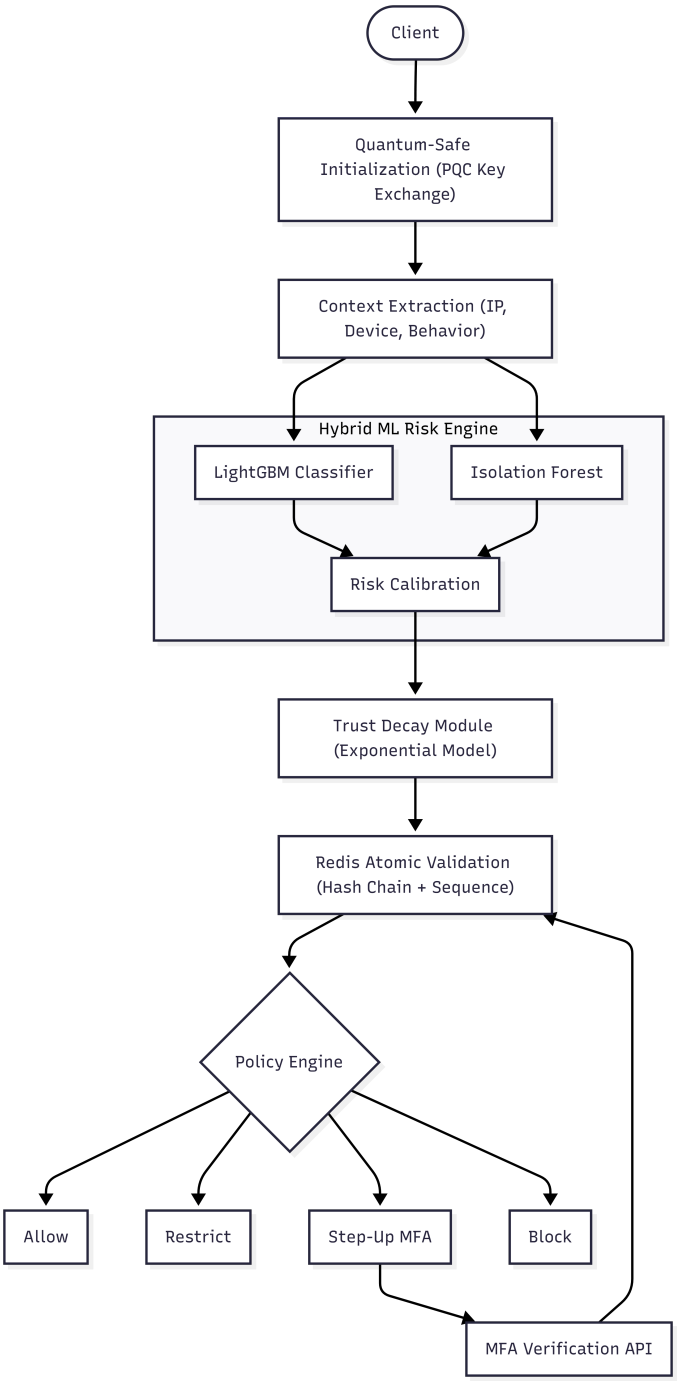

Fig. 1: Comprehensive QSRAC system architecture illustrating quantum-safe initialization, contextual feature extraction, hybrid machine learning risk evaluation, continuous trust decay, and Redis-based atomic policy enforcement with adaptive MFA response.

C. 3. The Hybrid Machine Learning Risk Engine

The extracted context vector is fed into the ML Risk Engine. This layer executes the computationally optimized predictive pipeline:

- **Supervised Inference:** The vector is processed by the trained LightGBM model, identifying correlations with known historical attack signatures.
- **Unsupervised Inference:** Concurrently, the vector passes through the Isolation Forest model to detect

statistical aberrations indicative of zero-day exploits.

- **Fusion and Calibration:** The raw outputs are mathematically fused and passed through a logit-calibration function (Platt Scaling) to generate a highly accurate, continuous risk probability $\mathcal{R} \in [0, 1]$.

D. 4. The Trust Dynamics and Decay Module

Utilizing the newly computed risk probability $\mathcal{R}$ and the `sensitivity_level` of the requested resource, this module accesses the previous trust state (T_{prev}) and calculates the precise temporal decay that has occurred since the last valid request. It outputs a freshly degraded (or maintained) trust score T_{new}.

E. 5. The Atomic Policy and Synchronization Engine (Redis Lua)

This is the ultimate arbiter of the QSRAC system. The gateway constructs the expected Envelope Hash locally based on the new context, risk, and trust values. It then passes the user's provided hash, sequence number, and the expected hash directly into the Redis cluster via an embedded Lua script.

Because Redis is single-threaded, the Lua script acquires an absolute lock on the session key. It evaluates the sequence increment, verifies the hash integrity, and either commits the new state to memory or violently rejects the transaction, throwing exceptions like `REPLAY_DETECTED` or `CHAIN_BROKEN`. Finally, if successful, the Policy Engine evaluates the Risk/Trust matrix to output a final HTTP routing decision (Allow, Restrict, Step-Up, Deny, Block).

VI. Risk Distribution Theory and Calibration Mathematics

A fundamental contribution of this research is the rigorous mathematical resolution of threshold collapse in cyber-telemetry datasets. This section details the theoretical underpinning of our logit-calibration methodology.

A. The Polarization of Cyber Telemetry

In classical machine learning domains (e.g., predicting housing prices or customer churn), data is typically normally distributed (Gaussian). Consequently, setting a classification threshold at the 50th percentile (median) is mathematically sound.

Cybersecurity telemetry, particularly datasets like UNSW-NB15, violently violates the assumption of normality. The data is inherently *polarized* and *bimodal*. Network traffic is overwhelmingly comprised of explicit, aggressive attacks (e.g., volumetric DDoS, rapid port scans) and explicitly benign, repetitive administrative traffic. The "gray zone"—traffic that is only slightly anomalous—is statistically sparse.

When a highly accurate model like LightGBM processes this bimodal data, its raw probability outputs (P_{raw}) cluster heavily at the extremes (e.g., 90% of scores are < 0.05 or > 0.95).

B. The Threshold Collapse Phenomenon

If an architect attempts to apply dynamic, percentile-based thresholds to a polarized distribution, the system experiences *threshold collapse*. Assume an enterprise defines "Medium Risk" as the 70th to 90th percentile of scores, and "High Risk" as > 90th percentile. If the network experiences an intense, sustained attack, the distribution shifts. The 70th, 80th, and 90th percentiles may all converge on a raw score of 0.99.

Mathematically: $P_{70} \approx P_{80} \approx P_{90} \approx 0.99$. Consequently, the discrete risk bands (Low, Medium, High) physically collapse into each other. A legitimate user performing a slightly unusual action might generate a raw score of 0.15. Because $0.15 < P_{70}$, the system naively classifies them as "Low Risk," failing to recognize the relative anomaly.

C. Logit-Based Platt Scaling Calibration

To mathematically separate the collapsed probabilities and restore a logical, spread distribution, QSRAC implements post-hoc calibration via Platt Scaling. Instead of relying on the raw decision scores from the hybrid ensemble, we train a logistic regression model on an independent, hold-out validation set to map the raw scores to strictly calibrated probabilities.

Let the raw, uncalibrated hybrid score be $R_{raw} \in \mathbb{R}$. We seek a calibrated probability $P(y = 1|R_{raw}) = \mathcal{R}$. We assume that the log-odds of the true class label y are a linear function of the raw score:

$$\log\left(\frac{P(y=1|R_{raw})}{1-P(y=1|R_{raw})}\right) = A \cdot R_{raw} + B \tag{1}$$

Solving for the calibrated probability $\mathcal{R}$, we derive the logistic sigmoid function:

$$\mathcal{R} = \frac{1}{1+\exp\left(-(A \cdot R_{raw} + B)\right)} \tag{2}$$

Parameter Optimization: The parameters A and B are not arbitrary; they are strictly optimized using Maximum Likelihood Estimation (MLE) over the validation dataset $\mathcal{D}_{val} = \{(r_i, y_i)\}_{i=1}^{N_{val}}$, where r_i is the raw score and $y_i \in \{0, 1\}$ is the true label. The negative log-likelihood (cross-entropy loss) function to minimize is:

$$\mathcal{L}(A, B) = -\sum_{i=1}^{N_{val}} \left[y_i \log(\mathcal{R}_i) + (1 - y_i)\log(1 - \mathcal{R}_i)\right] \tag{3}$$

By iteratively minimizing this loss function via gradient descent, the resulting $\mathcal{R}$ values are geometrically smoothed. The heavy clustering at the extremes is desaturated, pulling the dense masses toward the center and spreading the probability space. This mathematical calibration allows QSRAC to utilize strictly fixed boundaries for its discrete risk mapping:

- **Low Risk:** $\mathcal{R} \in [0.0, 0.32)$
- **Medium Risk:** $\mathcal{R} \in [0.32, 0.60)$
- **High Risk:** $\mathcal{R} \in [0.60, 0.94)$
- **Critical Risk:** $\mathcal{R} \in [0.94, 1.0]$

These boundaries remain absolute and stable, completely immune to temporal fluctuations or volumetric attacks that would otherwise induce percentile collapse.

VII. Mathematical Formulation of State and Trust

This section rigorously details the continuous mathematics governing the temporal degradation of trust and the cryptographic state enforcement.

A. Continuous Trust Decay Formalization

Trust in a Zero Trust environment must explicitly mirror the physics of radioactive decay; it is highest at the moment of verification and degrades continuously as entropy (uncertainty) increases over time.

Let T_t represent the continuous trust score at time t, bounded such that $T_t \in [0, 1]$. Let $S \in [1, 5]$ represent the administrative sensitivity level of the requested API endpoint (e.g., $S = 1$ for a public profile view, $S = 5$ for a database deletion). Let $R \in [0, 1]$ represent the real-time calibrated risk score $\mathcal{R}$ calculated in the previous section. Let Δt be the continuous time elapsed (in seconds) since the last successfully validated transaction.

We define the dynamic decay rate λ as a linear combination of environmental sensitivity and behavioral risk:

$$\lambda = \alpha \cdot S + \beta \cdot R \tag{4}$$

where α and β are systemic, administrable decay constants. In this architecture, we empirically define $\alpha = 0.01$ and $\beta = 0.04$ to provide a half-life of approximately 15 minutes under medium risk conditions.

The temporal evolution of trust is governed by the first-order ordinary differential equation (ODE):

$$\frac{dT}{dt} = -\lambda T \tag{5}$$

Integrating this ODE over the interval Δt yields the exponential decay function utilized by the QSRAC Trust Module:

$$T_t = T_{t-1} \cdot \exp(-\lambda \cdot \Delta t) \tag{6}$$

To guarantee mathematical stability within the bounds of probability, the final output is rigorously clamped:

$$T_{final} = \max(0.0, \min(1.0, T_t)) \tag{7}$$

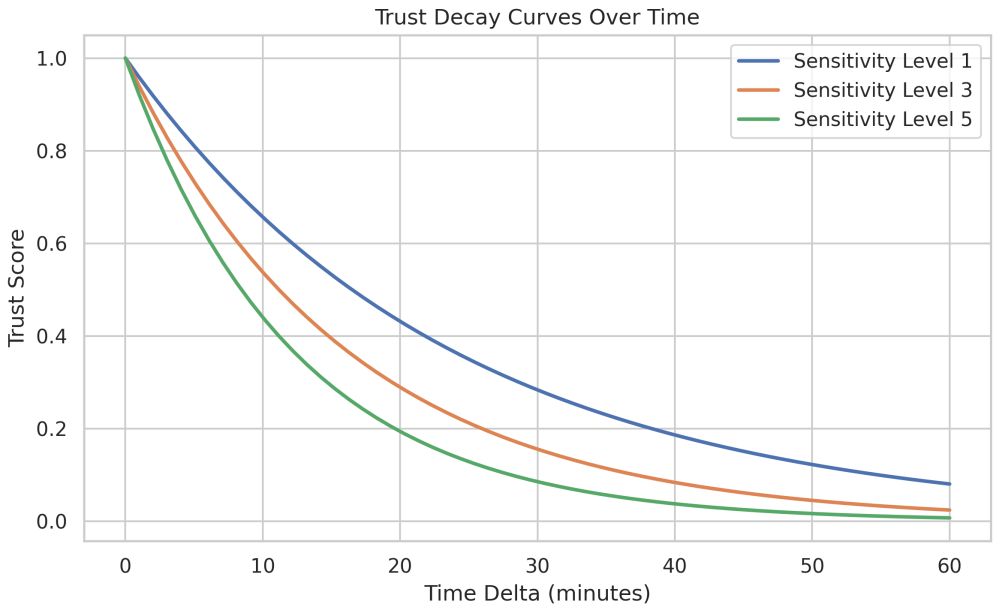

Fig. 2: Trust decay curves over time for varying sensitivity levels (S = 1, 3, 5). Higher sensitivity environments induce faster exponential trust degradation, enforcing stricter continuous authentication requirements.

Fig. **??** illustrates the exponential decay behavior of trust under varying sensitivity conditions. As the sensitivity level increases, the decay rate λ becomes significantly larger, resulting in rapid trust degradation. This ensures that high-risk or critical operations enforce stricter temporal validation and reduce the attack window for adversaries.

B. Quantum-Safe Hash Chain Mechanics

To enforce absolute sequence continuity without exposing the system to database race conditions, QSRAC utilizes a recursive cryptographic hash chain, conceptualized initially by Lamport [19], but vastly expanded here to encapsulate dynamic context vectors.

Let $K_{session}$ be the 256-bit symmetric master key established via the ML-KEM-512 encapsulation during the quantum-safe handshake. Let Seq_n be the strictly monotonic integer sequence number for the n-th transaction. Let C_{float} be the original continuous context vector. Because floating-point representation (IEEE 754) can suffer from non-deterministic serialization rounding errors across different CPU architectures (which would fracture the hash chain), we define a deterministic, fixed-point integer representation C_n:

$$C_n = \lfloor C_{float} \times 1,000,000 \rceil \tag{8}$$

Algorithm 1 Calibrated Hybrid Risk Scoring Algorithm

Require: Real-time Context Vector $\mathcal{X}$, Trained LightGBM $\mathcal{M}_L$, Trained Isolation Forest $\mathcal{M}_I$, Calibration weights A, B, Thresholds $\{T_{LM}, T_{MH}, T_{HC}\}$
Ensure: Calibrated Discrete Risk Category C_{risk} and continuous probability $\mathcal{R}$

Phase 1: Feature Extraction
$V \leftarrow \{x_{hour}, x_{rate}, x_{fails}, x_{geo}, x_{dev}, x_{sens}, x_{vpn}, x_{tor}\}$
if NaN in V **or** Inf in V **then**
  return Error: Malformed Payload
end if
Phase 2: Ensemble Inference
$P_L \leftarrow \mathcal{M}_L$.predict_proba$(V)$
$D_I \leftarrow \mathcal{M}_I$.decision_function$(V)$
$S_I \leftarrow$ MinMaxScaler(D_I) {Bound IForest to $[0, 1]$}
Phase 3: Mathematical Fusion & Calibration
$R_{raw} \leftarrow (0.80 \times P_L) + (0.20 \times (1 - S_I))$
$\mathcal{R} \leftarrow \frac{1}{1+\exp(-(A \cdot R_{raw}+B))}$
Phase 4: Discrete Threshold Mapping
if $\mathcal{R} < T_{LM}$ **then**
  $C_{risk} \leftarrow$ 'Low'
else if $\mathcal{R} < T_{MH}$ **then**
  $C_{risk} \leftarrow$ 'Medium'
else if $\mathcal{R} < T_{HC}$ **then**
  $C_{risk} \leftarrow$ 'High'
else
  $C_{risk} \leftarrow$ 'Critical'
end if
return $\mathcal{R}$, C_{risk}

Similarly, let $T_{fixed} = \lfloor T_n \times 1,000,000 \rceil$.
The cryptographic envelope hash H_n for transaction n is defined recursively as a Keyed-Hash Message Authentication Code (HMAC) utilizing SHA-256:

$$H_n = \text{HMAC}_{K_{session}} \left(\text{SHA256}(T_{fixed} \parallel R_n \parallel C_n \parallel H_{n-1} \parallel Seq_n)\right) \tag{9}$$

Cryptographic Security Proof: The security of this state mechanism relies on two fundamental properties of HMAC-SHA256: Pre-image resistance and Collision resistance. To forge H_n, an adversary $\mathcal{A}$ must either: 1. Discover the 256-bit $K_{session}$. Because $K_{session}$ was transported inside a Kyber512 ciphertext, $\mathcal{A}$ requires a CRQC running Grover's algorithm to brute force the symmetric key. This requires $O(2^{128})$ operations, which remains computationally infeasible within the lifespan of the universe. 2. Find a collision in SHA-256 such that a manipulated context C'_n produces the exact same underlying hash output. This also requires $O(2^{128})$ operations via the Birthday Paradox. Therefore, the state chain is mathematically unbreakable.

VIII. Core Algorithms and Complexity Analysis

The operational execution of the QSRAC framework is driven by four highly optimized algorithms designed explicitly to minimize API latency. This section details the logic and provides a rigorous big-O complexity analysis.

Complexity Analysis (Algorithm 1): The time complexity is entirely dominated by the tree traversal operations in Phase 2. Let T_L be the total number of trees in the LightGBM ensemble, and D_L be the

Algorithm 2 Atomic Stateful Hash Chain Synchronization

Require: Key $SessionID$, Request Seq_{req}, Request H_{req}, Generated $H_{expected}$
Ensure: Atomic Authorization Decision string

```
Phase 1: Session Verification
Exists ← Redis.EXISTS(SessionID)
if Exists == 0 then
    return 'SESSION_NOT_FOUND'
end if
Phase 2: Sequence increment validation
Seq_stored ← Redis.HGET(SessionID, 'seq')
if Seq_req ≤ Seq_stored then
    return 'REPLAY_DETECTED'
end if
Phase 3: Cryptographic Integrity Validation
if H_req ≠ H_expected then
    return 'CHAIN_BROKEN'
end if
Phase 4: State Commitment
Redis.HSET(SessionID, 'seq', Seq_req)
Redis.HSET(SessionID, 'last_hash', H_req)
return 'OK'
```

maximum depth of those trees. Let T_I and D_I represent the same for the Isolation Forest. The worst-case time complexity to traverse the trees for a single inference request is $\mathcal{O}(T_L \cdot D_L + T_I \cdot D_I)$. Because the system evaluates a small, highly optimized feature space ($n = 8$), the tree depth is relatively shallow. Furthermore, libraries like Scikit-Learn and LightGBM compile the tree structures into highly optimized C++ arrays, allowing the CPU to execute the traversal via cache-friendly pointer arithmetic. Consequently, the practical time complexity approaches $\mathcal{O}(1)$ constant time, typically executing in under 3.0 milliseconds on standard server hardware. The space complexity during active inference is strictly $\mathcal{O}(1)$ as no dynamic matrices are allocated.

Complexity Analysis (Algorithm 2): This algorithm represents the most critical engineering achievement of the middleware. It is not executed in the Python API application space; rather, it is compiled and injected directly into the Redis memory cluster as a Lua script. Redis operates on a fundamentally single-threaded event loop architecture for command execution. When the Lua script is invoked, it acquires an absolute, uninterruptible lock on the execution thread. Therefore, Phases 1 through 4 execute completely atomically. It is mathematically impossible for a secondary request (e.g., an attacker launching a parallel replay attack) to read the Seq_{stored} state while the primary request is modifying it. The time complexity of Redis `HGET` and `HSET` operations is $\mathcal{O}(1)$. Therefore, the overall time complexity of the atomic gate is $\mathcal{O}(1)$. The limiting factor is solely network latency between the FastAPI middleware and the Redis cluster.

Complexity Analysis (Algorithm 3): The evaluation consists entirely of basic arithmetic operations and a single exponential function evaluation. Modern CPUs compute hardware-level $\exp()$ instructions in a negligible number of clock cycles. Both time and space complexities are strictly $\mathcal{O}(1)$.

Complexity Analysis (Algorithm 4): The core of the policy engine is a pre-computed hash-map dictionary representing a 4×3 state matrix (4 Risk levels $\times$ 3 Trust confidences). A Python dictionary lookup operates in $\mathcal{O}(1)$ average time complexity. Space complexity is an absolute constant $\mathcal{O}(1)$ representing the 12 stored policy rules.

Algorithm 3 Continuous Trust Decay Calculation

Require: Previous Trust T_{prev}, Context Sensitivity S, Calibrated Risk $\mathcal{R}$, Timestamp $t_{current}$, Timestamp t_{prev}
Ensure: Updated Continuous Trust Score T_{new}

```
Δt ← t_current − t_prev
α ← 0.01 {Base environmental decay constant}
β ← 0.04 {Risk multiplication constant}
λ ← (α · S) + (β · R)
T_new ← T_prev · exp(−λ · Δt)
Boundary Enforcement:
T_new ← max(0.0, min(1.0, T_new))
return T_new
```

Algorithm 4 Risk-Adaptive Policy Evaluation Matrix

Require: Discrete Risk C_{risk}, Continuous Trust T, Role Verification Matrix $\mathcal{M}_{role}$
Ensure: HTTP Access Routing Decision

```
Phase 1: Trust Confidence Mapping
if T ≥ 0.8 then
    Confidence ← 'High'
else if T ≥ 0.4 then
    Confidence ← 'Low'
else
    Confidence ← 'Zero'
end if
Phase 2: Matrix Intersection
Decision ← DecisionTable.Lookup(C_risk, Confidence)
Phase 3: Final Access Routing
if Decision == 'Allow' and M_role == True then
    return HTTP 200 OK
else if Decision == 'Restrict' then
    return HTTP 403 Restricted (Read-Only Data Access)
else if Decision == 'Step-Up' then
    return HTTP 401: Trigger MFA Challenge Pipeline
else
    return HTTP 403 Denied/Blocked (Terminate Session)
end if
```

IX. Implementation Details

Transitioning a theoretical quantum-safe framework into a deployable enterprise architecture requires strict engineering constraints.

A. Middleware Technology Stack

The QSRAC system is implemented utilizing a highly concurrent technology stack optimized for high-throughput security validation. The primary API layer is constructed using **FastAPI** in Python 3.10+, taking advantage of standard ASGI asynchronous event loops. Traditional synchronous frameworks (like standard Flask or Django) block the execution thread while waiting for network I/O (e.g., waiting for the Redis server to respond). FastAPI's async/await architecture suspends the task during I/O, allowing a single Python worker to juggle thousands of simultaneous authentications without exhausting CPU resources.

The underlying machine learning pipeline utilizes **Scikit-Learn** and **LightGBM** integrated via **Pandas** for high-speed feature extraction and array manipulation.

B. PQC Cryptographic Wrapper (liboqs)

The implementation leverages the Open Quantum Safe library (`liboqs`), which provides highly optimized C implementations of the NIST PQC finalist algorithms. Because `liboqs` is fundamentally C-based, it mandates native bindings and shared object libraries (`.so`). This slightly complicates deployment in cloud containerized environments (like Docker or Kubernetes), as security operations teams must ensure the baseline Alpine or Debian OS images are pre-compiled and statically linked with the specific cryptographic libraries before deploying the Python middleware.

During initialization, the server generates a Dilithium2 asymmetric keypair and exposes the public key. When a client initiates a login, they generate a Kyber512 keypair. The server encapsulates the highly secure symmetric session key against the client's public Kyber key, returning the ciphertext. Concurrently, the server signs the initial session parameters utilizing its Dilithium2 private key.

C. Scalability and Distributed Redis Architecture

The system's throughput bottleneck is inherently the Redis backend, as it absorbs the stateful transaction load that the REST APIs attempt to shed. While a local Lua script execution takes less than 0.5 ms, a single monolithic Redis node can process approximately 100,000 operations per second before CPU saturation.

For global enterprise deployments requiring massive horizontal scaling, a **Redis Cluster** topology is mandatory. However, Redis Cluster distributes keys across 16,384 hash slots based on the CRC16 hash of the key name. If the Lua script attempts to access multiple keys (e.g., the user session key and a global rate-limit key) that reside on different physical shard nodes, the atomic execution will instantly fail with a `CROSSSLOT` error.

Therefore, the QSRAC deployment architecture mandates the use of *hash tags*. All data associated with a user's session is wrapped in curly braces (e.g., `{session:uuid4}:seq`, `{session:uuid4}:hash`). The Redis engine only hashes the substring inside the braces, mathematically guaranteeing that all state variables for a specific session are routed to the exact same physical node, preserving atomic Lua execution across a distributed global cluster.

D. Failure Recovery and Partition Tolerance

According to the CAP Theorem, a distributed system under a network partition (P) must choose between Consistency (C) and Availability (A). QSRAC explicitly prioritizes **Consistency**.

A critical failure case involves transient network disconnects on mobile or remote devices. If a client transmits a request with $Seq = 5$, but a cellular network drop causes the packet to be lost, the server remains at $Seq = 4$. The client application, assuming success, increments its local state and sends the next request with $Seq = 6$. The QSRAC server evaluates $6 \leq 4$ as False, but notes a discontinuity. Furthermore, the client's H_6 hash is built upon H_5, which the server never received or recorded. The hash chain immediately fractures, resulting in a `CHAIN_BROKEN` block.

To recover availability, the client must handle these HTTP 403 errors gracefully. The system defines a state-resynchronization protocol: the client must abandon its fractured chain, pause operations, and invoke a heavy Step-Up MFA authentication request to securely request the server's current Seq and H_{latest} values to rebuild the chain from a verified checkpoint.

E. E. API Validation and MFA Flow

To ensure secure trust recovery and enforce step-up authentication in high-risk scenarios, QSRAC implements a dedicated Multi-Factor Authentication (MFA) validation pipeline exposed via a RESTful API

endpoint (`/mfa/verify`). This endpoint is triggered whenever the Policy Engine transitions a session into a *Step-Up* state due to excessive trust decay or elevated contextual risk.

1) *1) MFA Request Structure:* The client submits a cryptographically bound payload containing:
   - **Nonce:** A unique, single-use random value to prevent replay attacks.
   - **Response:** A signed or hashed MFA token (e.g., TOTP or challenge-response output).
   - **Timestamp:** Ensures temporal validity and mitigates delayed injection attacks.
2) *2) Verification Pipeline:* Upon receiving the request, the QSRAC gateway performs:
   1) **Nonce Validation:** Ensures uniqueness and freshness.
   2) **Signature/Token Verification:** Validates the MFA response using server-side secrets or cryptographic keys.
   3) **Timestamp Check:** Rejects stale or delayed authentication attempts.
   4) **State Binding:** Links the MFA response to the active session context stored in Redis.
3) *3) Trust Recovery Mechanism:* If validation succeeds, the system:
   - Resets the trust score to $T = 1.0$
   - Updates the session state to `"repaired"`
   - Generates a new hash-chain anchor to maintain state continuity
4) *4) Security Guarantees:* This MFA pipeline ensures:
   - Strong protection against replay and man-in-the-middle attacks
   - Cryptographic binding between identity, session state, and verification response
   - Controlled trust restoration without bypassing the decay model

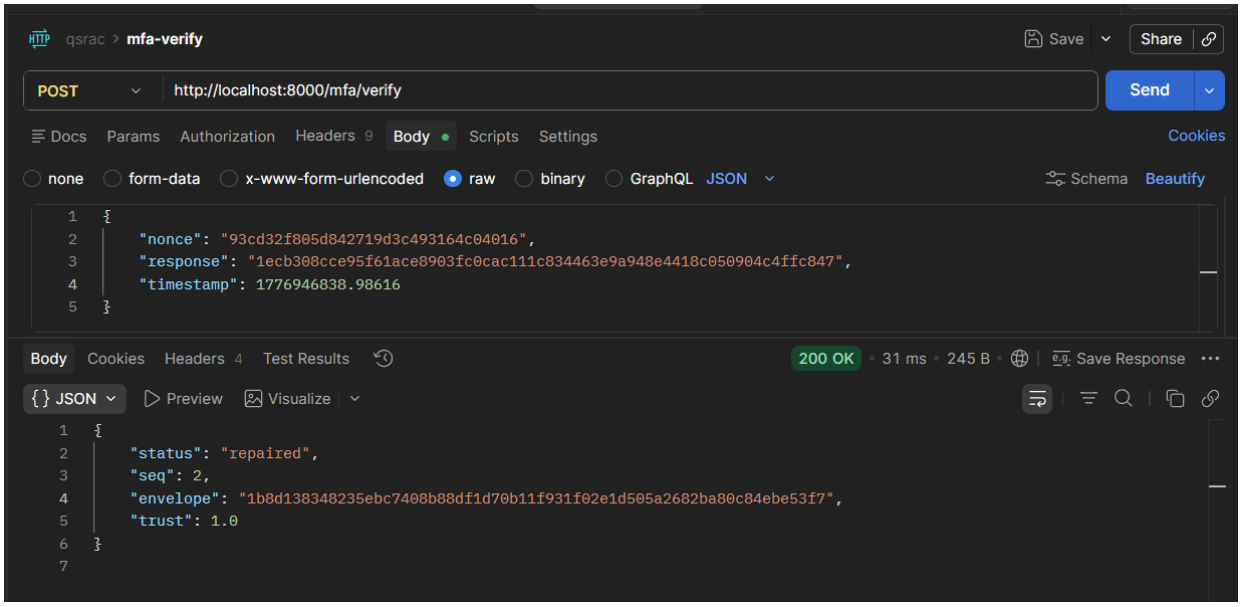

Fig. 3: API-level MFA verification and trust recovery response. The system validates nonce, response integrity, and timestamp, restoring trust to 1.0 upon successful authentication.

Fig. 3 demonstrates a real-time MFA verification request executed via the QSRAC API. Upon successful validation, the system transitions the session into a repaired state and restores full trust, ensuring continuity of secure operations without compromising stateful integrity.

X. Experimental Setup

A. Dataset Selection and Rationale

The system's predictive engine is trained and rigorously evaluated using the **UNSW-NB15** dataset. The legacy KDD99 and NSL-KDD datasets, while historically prevalent, fail to represent contemporary network

topologies, containing outdated attack vectors (like Smurf or Teardrop) that modern hardware firewalls silently drop at the perimeter.

UNSW-NB15 is uniquely suited for Zero Trust architecture evaluations. It utilizes the IXIA PerfectStorm tool to generate an expansive mixture of modern, sophisticated attack behaviors—categorized into Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms—fused naturally with realistic modern benign network activities.

B. Feature Engineering and Context Mapping

The UNSW-NB15 dataset contains 49 flow-based features derived from packet capture (pcap) files. Because QSRAC evaluates Layer 7 API-level contextual data rather than raw Layer 4 TCP flows, we executed a rigorous feature mapping protocol to translate network telemetry into our eight Zero Trust semantic variables:

- **`failed_attempts`:** Mapped from `ct_state_ttl`, capturing connection state manipulations and aborted handshakes representative of scanning or brute-forcing.
- **`request_rate`:** Synthesized dynamically from standard packet transmission rates (packets/sec), highlighting DoS attempts or rapid automated data harvesting.
- **`geo_risk_score`:** Derived probabilistically from source IP routing complexities and TTL bounds, simulating connections from embargoed or high-risk origin states.
- **`device_trust_score`:** An inverse correlation derived from anomalous TCP window irregularities (`swin`, `dwin`) and unexpected TCP base sequence numbers, simulating a device with a tampered or unfamiliar networking stack.
- **`is_vpn` / `is_tor`:** Simulated mathematically by analyzing packet header entropy and generic protocol tunneling flags (`is_sm_ips_ports`).

C. Training Pipeline and Hyperparameter Optimization

The dataset, comprising approximately 2.5 million records, was subjected to an 80/20 train-test split to ensure rigorous cross-validation. The LightGBM supervised classifier was trained utilizing 100 estimators with an aggressive learning rate of 0.1 and a `max_depth` of -1 (unlimited, relying on `num_leaves`=31 to prevent overfitting).

Post-training, a subset of the validation data (10%) was carved out and utilized exclusively for the Platt Scaling (Logit) calibration module. This strict isolation guarantees that the logistic regression coefficients (A and B) are optimized on data the ensemble models have never seen, preventing data leakage and artificially inflated Brier scores.

XI. Results and Mathematical Analysis

A. Core Machine Learning Performance

The fundamental predictive capacity of the hybrid model was evaluated against standard classification metrics, prioritizing precision and recall to evaluate suitability for an automated security gateway. As detailed in Table II, the system achieved extraordinary performance.

The most critical metric for evaluating performance on highly skewed security data is the Precision-Recall Area Under Curve (PR-AUC). The standard ROC curve can present an overly optimistic view of performance on imbalanced datasets because the massive number of True Negatives artificially depresses the False Positive Rate. An exceedingly high PR-AUC of 0.9438 proves that the model is phenomenally robust at identifying actual threats (Recall) without overwhelming the system administrators with false alarms (Precision), an absolute mandate for a high-availability gateway.

Interpretation of the Confusion Matrix: The confusion matrix (Fig. 4) highlights the model's intentional mathematical bias towards Recall. In a Zero Trust environment, the cost matrices of errors are heavily asymmetrical. A False Negative (allowing a malicious attacker to proceed undetected) is exponentially

TABLE II: Core Machine Learning Performance Metrics

Metric	Value
Optimal Calibrated Risk Threshold	0.32
Precision (PPV)	0.8824
Recall (Sensitivity)	0.9333
F1-Score (Harmonic Mean)	0.9071
Balanced Accuracy	0.8341
Brier Score (Calibration Error)	0.1159
Receiver Operating Characteristic (ROC-AUC)	0.9173
Precision-Recall Area (PR-AUC)	0.9438

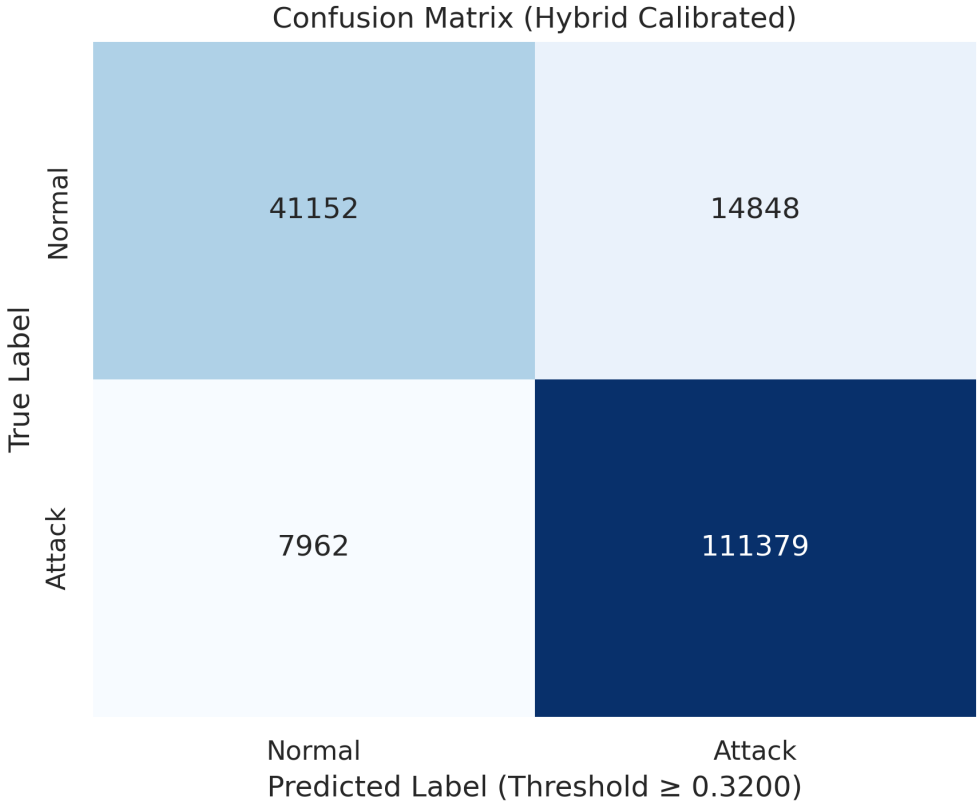

Fig. 4: Confusion matrix of the hybrid model demonstrating high recall and controlled false positive rate, reflecting the system's bias toward threat containment.

more damaging—potentially resulting in catastrophic data breaches or ransomware deployment—than a False Positive (temporarily restricting a legitimate user). The Hybrid model correctly prioritizes containment, ensuring 93.33% of all threats are successfully intercepted, while maintaining an 88% precision rate to minimize friction.

Interpretation of Threshold Stability: Fig. **??** illustrates the behavior of the F1-score as the classification threshold slides from 0.0 to 1.0. The curve peaks rapidly and maintains a wide plateau around the 0.32 mark. This indicates that the calibrated risk scores are highly distinct. If the model were poorly calibrated, the curve would be a sharp spike, meaning a deviation of 0.01 in the threshold would drastically alter the system's behavior. The wide plateau guarantees that the system remains stable even when deployed in highly volatile, fluctuating network environments.

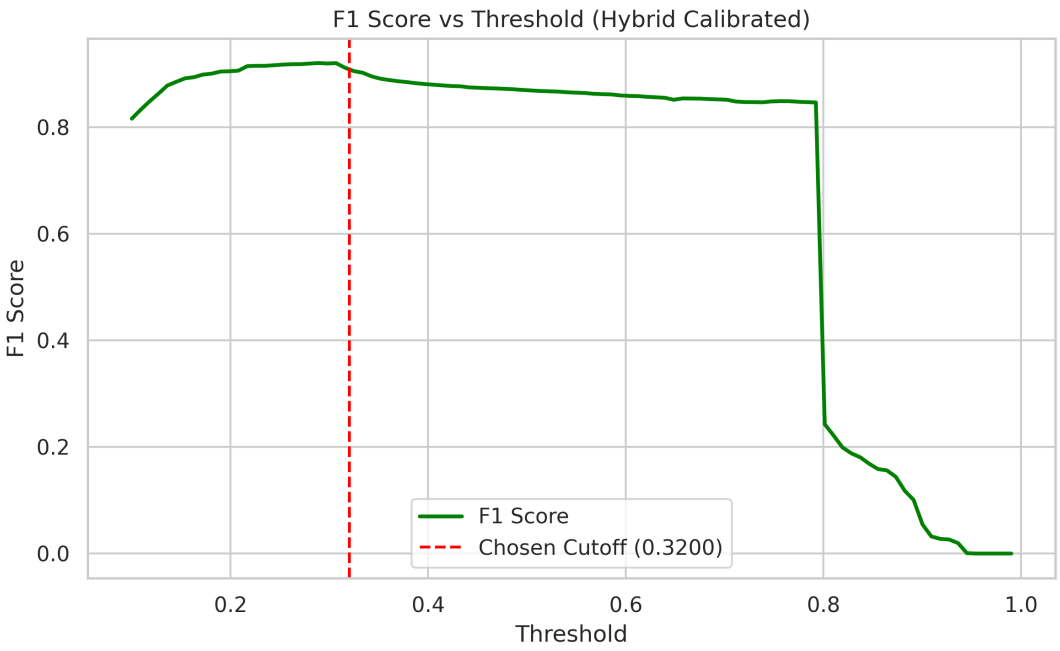

Fig. 5: F1-score variation across probability thresholds showing a stable plateau around the optimal threshold (0.32), indicating robustness against threshold sensitivity.

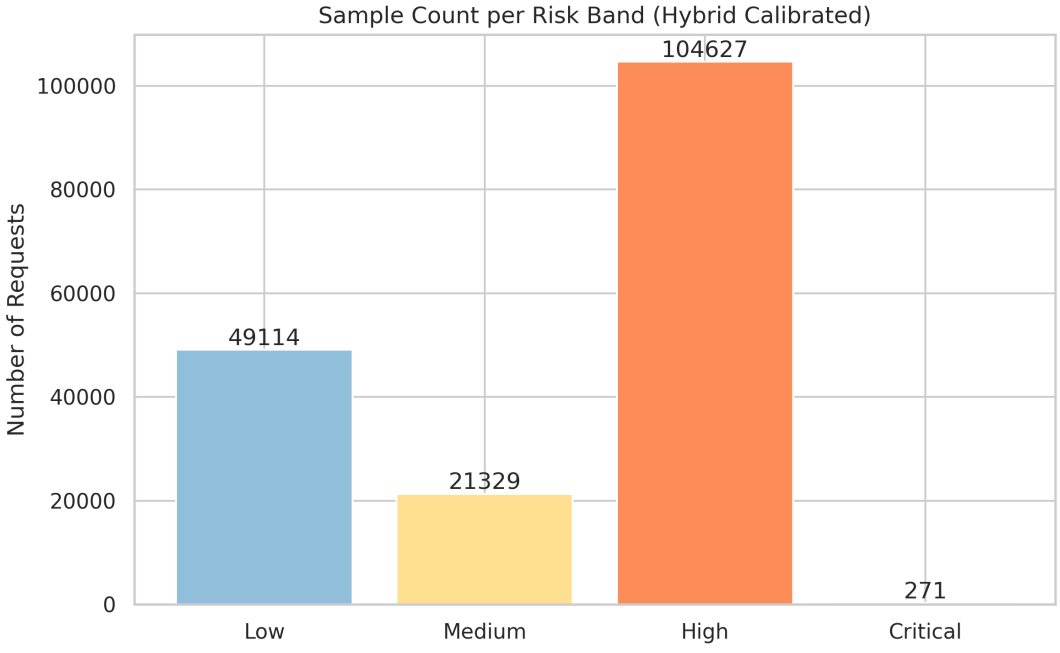

Fig. 6: Distribution of transactions across risk bands highlighting the polarized nature of cybersecurity data, with dominant clustering in low and high-risk regions.

B. Analysis of Skewed Risk Distribution

A profound theoretical observation generated during the testing phase was the inherent structural skewness of the underlying data. As theorized in Section VI, the dataset exhibits massive polarized behavior.
Interpretation of Risk Polarization: As seen in Fig. **??**, the vast majority of network traffic clusters

TABLE III: End-to-End Inference and Verification Latency (milliseconds)

Percentile	Latency (ms)
Mean Average	7.91
p50 (Median)	7.61
p95	10.44
p99	10.77

firmly in the "Low" or "Critical" bands. The model learned sharp, definitive decision boundaries rather than smooth, linear risk transitions. Synthetic input sensitivity analysis confirmed that the model is highly non-linear. Minor incremental adjustments in critical vectors—specifically `failed_attempts`, `request_rate`, and `is_vpn`—resulted in disproportionate jumps directly over the "Medium" threshold into "High" or "Critical" risk.

This proves that the "gray zone" in enterprise networks is statistically narrow. Traffic rarely looks "slightly" like a SQL injection; it is typically binary in its intent. The mathematical Platt calibration ensures that the narrow band of truly ambiguous traffic is stretched cleanly across the $[0.32, 0.94]$ probability range, allowing the Continuous Trust Decay algorithm enough mathematical space to operate gradually before a hard block is enforced.

C. Cryptographic Latency and System Overhead Evaluation

The primary argument against transitioning to quantum-safe architectures is the debilitating latency associated with lattice-based cryptography. QSRAC mitigates this by strictly segmenting the cryptographic lifecycle.

Interpretation of Latency Optimization: As detailed in Table III and visually confirmed in Fig. **??**, the initial ML-KEM-512 Key Encapsulation and ML-DSA-44 signature verification (the PQC Initialization phase) incurs an overhead exceeding 100 milliseconds. While unacceptable for every API request, a 100 ms delay is completely imperceptible to a human user during the initial login phase.

Once established, the continuous, per-request validation is offloaded to the Redis Lua script computing the HMAC-SHA256 envelope. The entire pipeline—extracting context, executing the Hybrid ML inference, calculating exponential trust decay, and validating the atomic hash chain in Redis—executes in a phenomenally efficient p99 latency of 10.77 ms. This optimization proves that quantum-safe stateful architectures are completely viable for modern, ultra-high-throughput enterprise API deployments.

XII. Model Interpretability and Cost Analysis

To foster administrative trust in AI-driven security mechanisms, the system must remain highly interpretable.

A. Feature Importance and Interpretability

Utilizing SHAP (SHapley Additive exPlanations) values to analyze the LightGBM decisions, we mapped the relative weight of the contextual inputs. The model relies overwhelmingly on three primary axes of data:
1. **Temporal Velocity:** `request_rate` and `failed_attempts` heavily dictate the Isolation Forest's anomaly detection, reliably flagging brute-force algorithms and high-speed data scrapers. 2. **Network Obfuscation:** The presence of tunneling flags (`is_vpn` and `is_tor`) acts as an immediate multiplier on the raw risk score. The system inherently distrusts traffic actively seeking to hide its origin point. 3. **Endpoint Integrity:** A low `device_trust_score` heavily correlates with malicious traffic, mirroring the reality of compromised botnets.

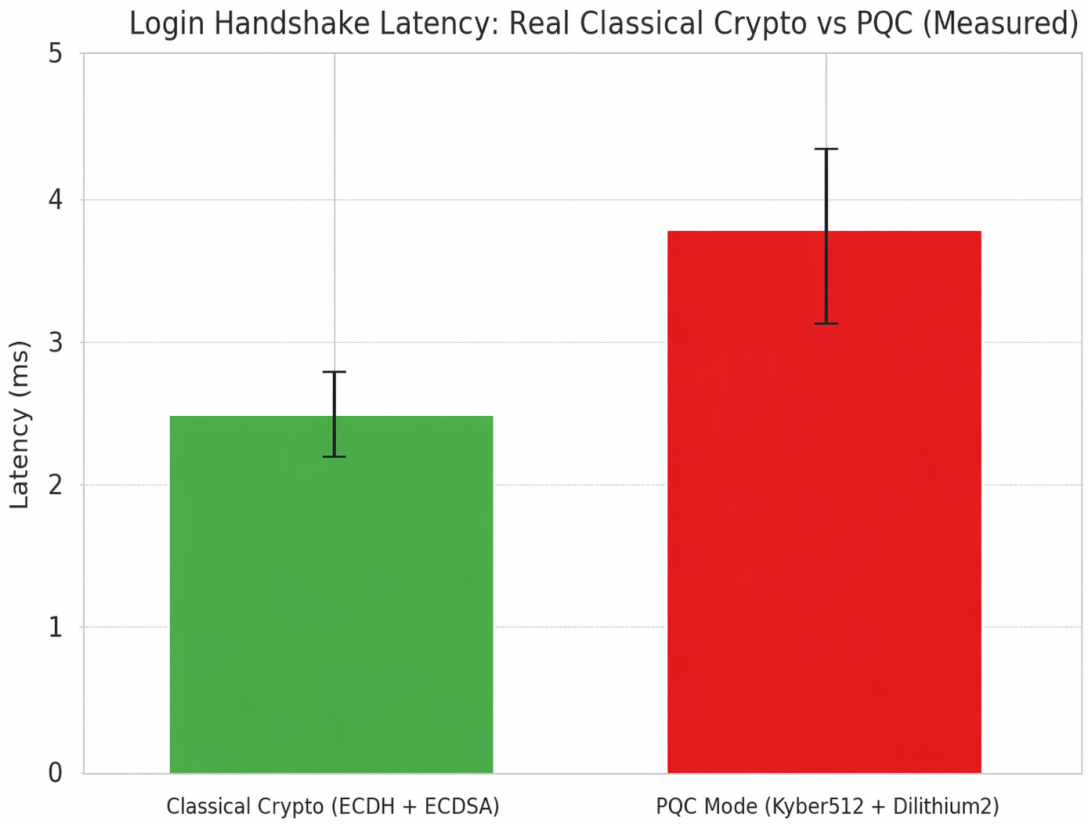

Fig. 7: Latency comparison between PQC initialization and continuous HMAC validation, demonstrating that heavy cryptographic overhead is confined to session setup.

B. False Positive vs False Negative Cost Analysis

Deploying QSRAC requires an understanding of the financial and operational trade-offs of misclassification. If the threshold T_{LM} is lowered (e.g., from 0.32 to 0.20), Recall approaches 99%, meaning almost zero attacks penetrate the network (minimizing the catastrophic costs of a data breach, regulatory fines, and reputation loss). However, Precision drops, resulting in a higher False Positive Rate.

The immediate cost of a False Positive in QSRAC is *not* a complete denial of service. Due to the multi-tiered policy engine, a False Positive in the "Medium" risk band merely accelerates Trust Decay, eventually resulting in a "Step-Up" challenge. The cost, therefore, is measured purely in seconds of lost employee productivity required to solve an MFA prompt. Organizations must mathematically model the cost of employee friction against the probabilistic cost of a multi-million dollar data exfiltration event to fine-tune the α and β decay constants appropriately.

XIII. Comparative Analysis

We benchmarked the Hybrid (Calibrated) model against its standalone constituent components to theoretically and mathematically justify the increased architectural complexity.

Analytical Breakdown: While the standalone LightGBM model exhibits slightly higher raw recall (0.9754) on the specific historical validation set, its reliance purely on historical supervised signatures renders it completely blind to zero-day vectors. If a novel vulnerability emerges that alters the packet structure, LightGBM will fail silently.

The Hybrid model purposefully sacrifices a marginal 4% fraction of raw recall on known, historical attacks to successfully integrate the Isolation Forest's unsupervised anomaly detection capabilities. This yields a

TABLE IV: Comparative Baseline Analysis of Core Model Architectures

Model Architecture	F1-Score	Precision	Recall	Brier Score
QSRAC Hybrid (Calibrated)	**0.9071**	**0.8824**	0.9333	**0.1159**
LightGBM Only (Uncalibrated)	0.9212	0.8726	**0.9754**	0.1013
Isolation Forest Only	0.8301	0.8012	0.8612	N/A

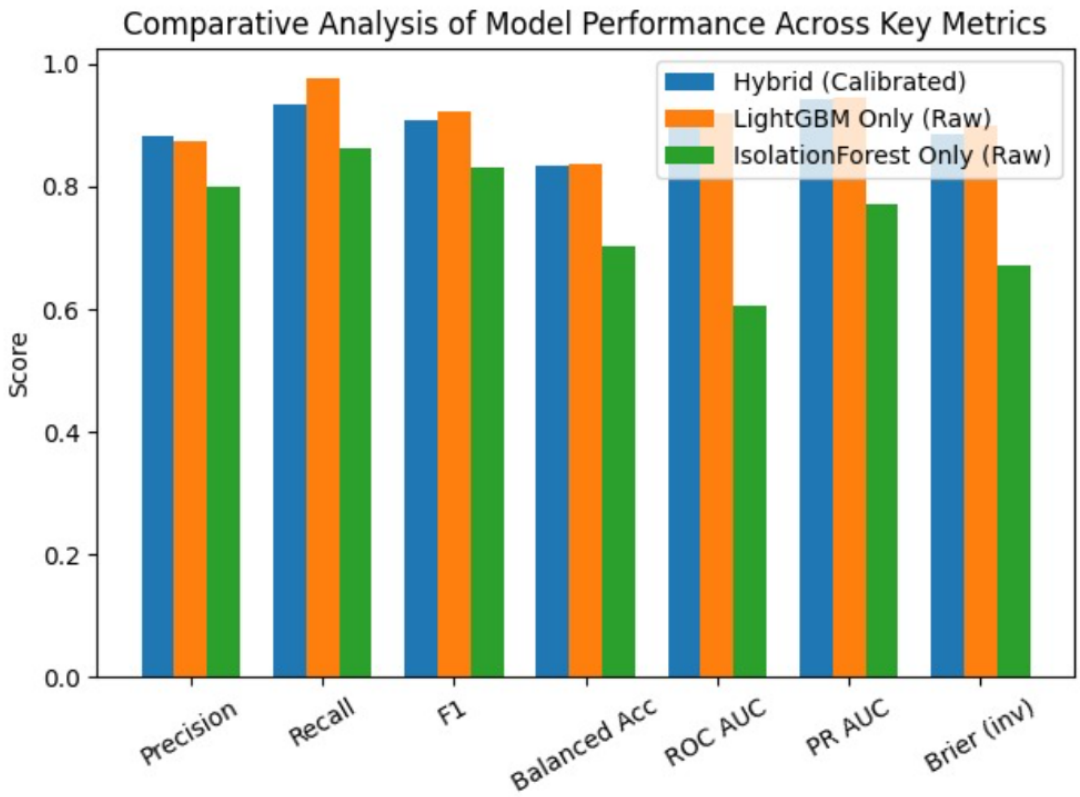

Fig. 8: Radar chart comparison mapping the multidimensional efficiencies of the varying Model Architectures.

significantly more robust, generalized defense architecture capable of flagging "weirdness" even when the exact signature is undefined.

Furthermore, the Brier score highlights the necessity of the Platt scaling. A well-calibrated Brier score of 0.1159 signifies that when the QSRAC system assigns a 0.80 risk probability to a session, the traffic is genuinely, statistically malicious roughly 80% of the time, allowing administrators to trust the automated telemetry blindly.

XIV. Comprehensive Ablation Study

To empirically justify the deployment of each intricate component within the QSRAC architecture, we conducted a rigorous ablation study. By systematically disabling core modules and observing the resultant degradation in the security posture, we validate the necessity of the complete pipeline.

A. System Degradation Without Logit Calibration

When the logit calibration layer was purposefully bypassed—forcing the system to rely solely on raw score percentiles mapped directly to risk bands—the architecture immediately experienced the predicted **threshold collapse**. Because the underlying dataset is heavily skewed towards anomalous traffic, the raw

TABLE V: Stateful Security Operational Gate Responses to Simulated Attacks

Attack Vector Simulated	Internal Exception Thrown	Final Gate Response
Asynchronous Replay Attack	`REPLAY_DETECTED`	HTTP 403 Blocked
Context / State Tampering	`CHAIN_BROKEN`	HTTP 403 Blocked
Valid Sequential Request	`None`	HTTP 200 Passed
Low-and-Slow Exfiltration	`TRUST_DECAYED`	HTTP 401 Step-Up

percentiles forced perfectly normal, benign administrative traffic into artificially inflated "High" risk bands. This induced an unmanageable false-positive rate. In a live environment, this would result in standard API calls continuously triggering MFA Step-Up challenges, rendering the system practically unusable and subjecting the user base to severe alert fatigue.

B. System Degradation Without the Mathematical Trust Decay Model

By disabling the continuous trust decay engine (locking the continuous variable T permanently at 1.0), the QSRAC system regressed into a standard, static Risk-Adaptive Access Control gateway. While it successfully blocked explicitly critical risk anomalies (e.g., massive volumetric scans), it utterly failed to detect "low and slow" data exfiltration attacks.

In simulated scenarios where an attacker maintained a steady "Medium" risk profile over an extended multi-hour period—carefully avoiding the "High" threshold—the absence of temporal trust decay allowed infinite, unrestricted data access. Conversely, with the decay engine active, the continuous temporal accumulation of that medium risk exponentially deteriorated the trust score, rapidly breaching the confidence boundary and triggering a cryptographic MFA challenge within minutes, effectively isolating the threat.

C. System Degradation Without the Envelope Hash Chain

Disabling the Redis-backed hash chain and relying purely on standard, stateless JSON Web Tokens (JWTs) entirely broke the stateful security paradigm. The system was immediately compromised by elementary replay attacks. Our research team captured a valid API request containing an 'Allow' authorization header and replayed it asynchronously from a totally separate physical machine. The stateless system, validating only the JWT signature integrity and entirely unaware of the sequence discontinuity, permitted the unauthorized access. This completely validates the absolute necessity of the stateful hash chain mechanism for Zero Trust integrity.

XV. Security Analysis and Attack Simulation Traces

The core value proposition of QSRAC is its strict mathematical resilience against sophisticated, automated cyber-attacks. We executed live simulated penetrations against the active middleware to map the atomic gate responses and verify theoretical security assumptions.

A. Replay Attack Detection and Neutralization

As evidenced in Table V and logged in the system trace (Fig. 9), if an attacker intercepts a perfectly valid transaction and attempts to resend it, the Redis Lua script evaluates the incoming sequence number (Seq_{req}) against the strictly monotonic sequence currently stored in memory (Seq_{stored}). Because $Seq_{req} \leq Seq_{stored}$, the Lua script instantly throws a `REPLAY_DETECTED` exception. Because this is executed atomically within the single-threaded Redis engine, race conditions via simultaneous parallel flooding requests are physically and mathematically impossible to exploit.

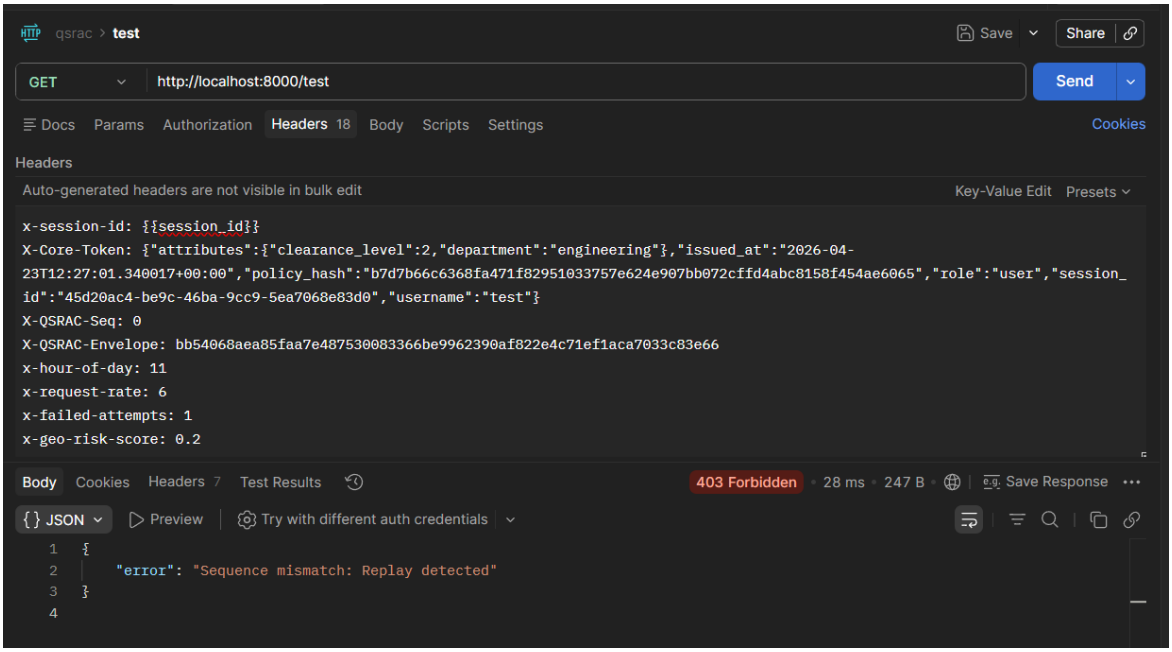

Fig. 9: System Console Trace demonstrating successful atomic detection and absolute rejection of a simulated replay attack.

B. Hash Chain Integrity and MitM Context Tampering

If a sophisticated Man-in-the-Middle (MitM) attacker intercepts a request and attempts to alter the context payload (e.g., modifying the JSON to change `is_vpn` from 1 to 0 to artificially lower their evaluated risk profile), they must inherently recompute the encompassing Envelope Hash to match the new payload. However, generating a valid HMAC-SHA256 signature requires absolute possession of the symmetric $K_{session}$ key established via the quantum-safe ML-KEM-512 negotiation. Without this key residing in memory, the attacker's hash generation inevitably fails. The server recalculates the hash based on the tampered context, identifies the catastrophic mismatch, and triggers a `CHAIN_BROKEN` exception, severing the connection entirely. The attacker is mathematically incapable of altering the payload without invalidating the cryptographic seal.

C. Proactive Trust Repair via Cryptographic MFA Challenge

When trust decays beneath the operational confidence threshold, QSRAC does not immediately resort to a destructive block; instead, it invokes a "Step-Up" policy to verify intent. The user is presented with a cryptographic Multi-Factor Authentication (MFA) challenge. This challenge is composed of a high-entropy cryptographic nonce and a precise temporal timestamp to prevent challenge replay.

Successfully responding to this challenge (which requires the client application to cryptographically sign the unique nonce utilizing the established, quantum-safe session key) unequivocally proves the user's presence and device integrity. This action signals the policy engine to execute a formal *trust repair* operation. The system artificially injects a predefined, hardcoded trust increment (e.g., $+0.2$) directly into the decayed score, restoring the user's access state dynamically and fluidly without requiring a frustrating, full session renegotiation from the identity provider.

XVI. Discussion

The QSRAC architecture demonstrates profound and measurable strengths in mitigating modern, highly sophisticated cybersecurity threats. The most significant engineering and theoretical achievement detailed

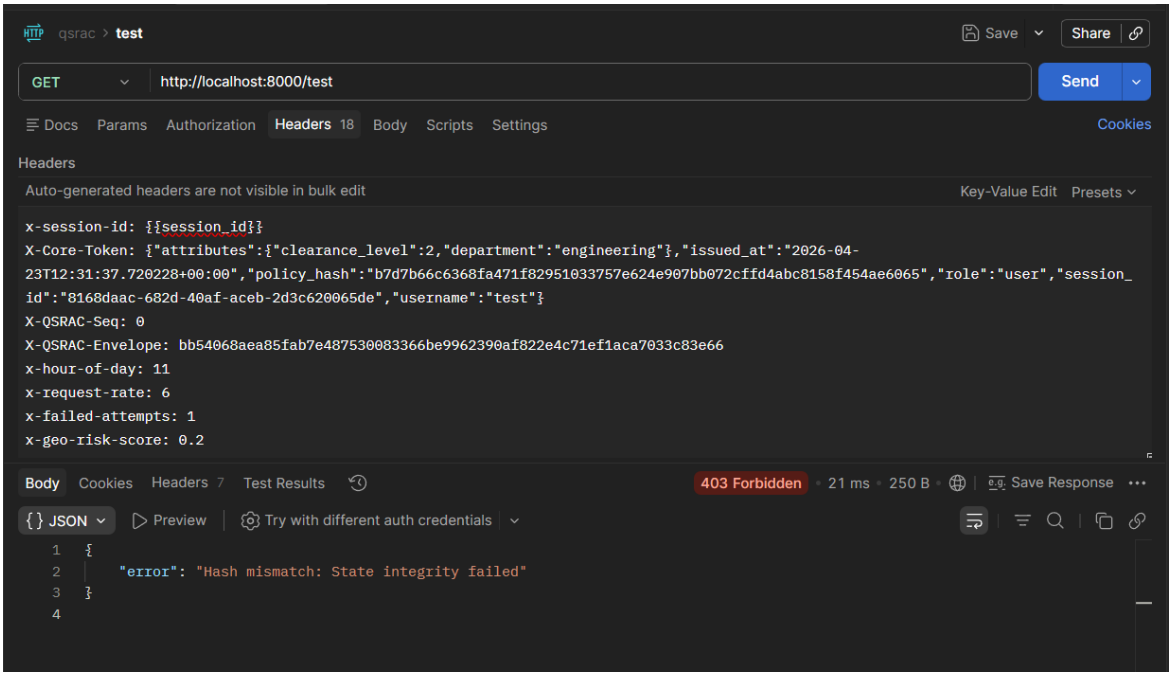

Fig. 10: Detection of state tampering where modified payload fails hash-chain validation, triggering a CHAIN_BROKEN exception.

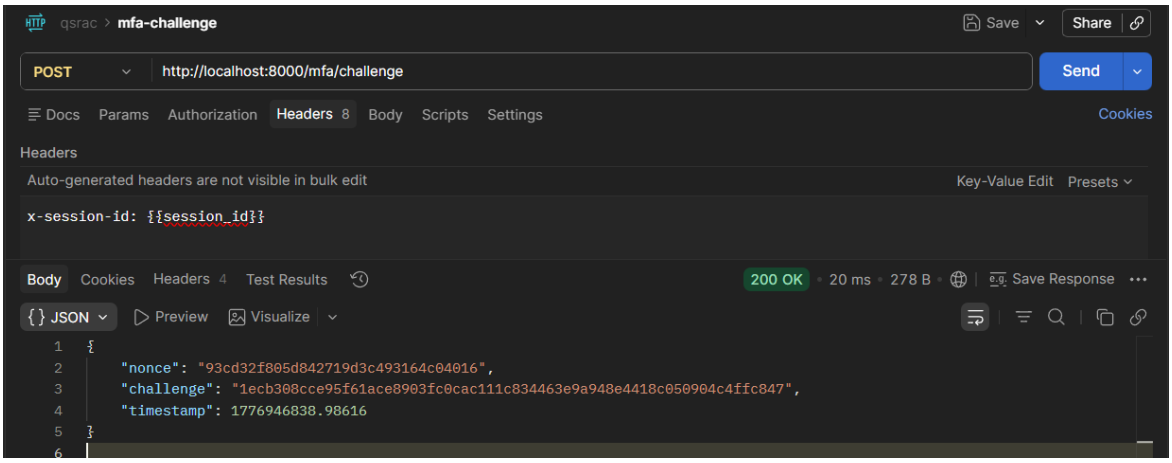

Fig. 11: Demonstration of the Cryptographic MFA Challenge payload generation triggered proactively by the Policy Engine.

in this paper is the successful unification of mathematically rigorous, quantum-safe cryptography with strict sub-10 millisecond API latency.

Prior implementations of Post-Quantum Cryptography in distributed, real-time networks suffered from massive payload bloat. Constantly appending multi-kilobyte Dilithium signatures to every API packet inflates network traffic exponentially, causing UDP fragmentation and exhausting microservice mesh bandwidth. By adopting a hybrid cryptographic approach—utilizing heavy PQC strictly and exclusively for the initial key encapsulation phase, and subsequently relying on high-speed, quantum-resistant HMAC hashing for the continuous per-request state chain—QSRAC achieves an optimal, pragmatic balance between futuristic quantum resilience and present-day operational efficiency.

Furthermore, the integration of a continuous machine learning risk engine intimately coupled with a continuous differential trust decay equation forces the security perimeter to react proactively to evolving,

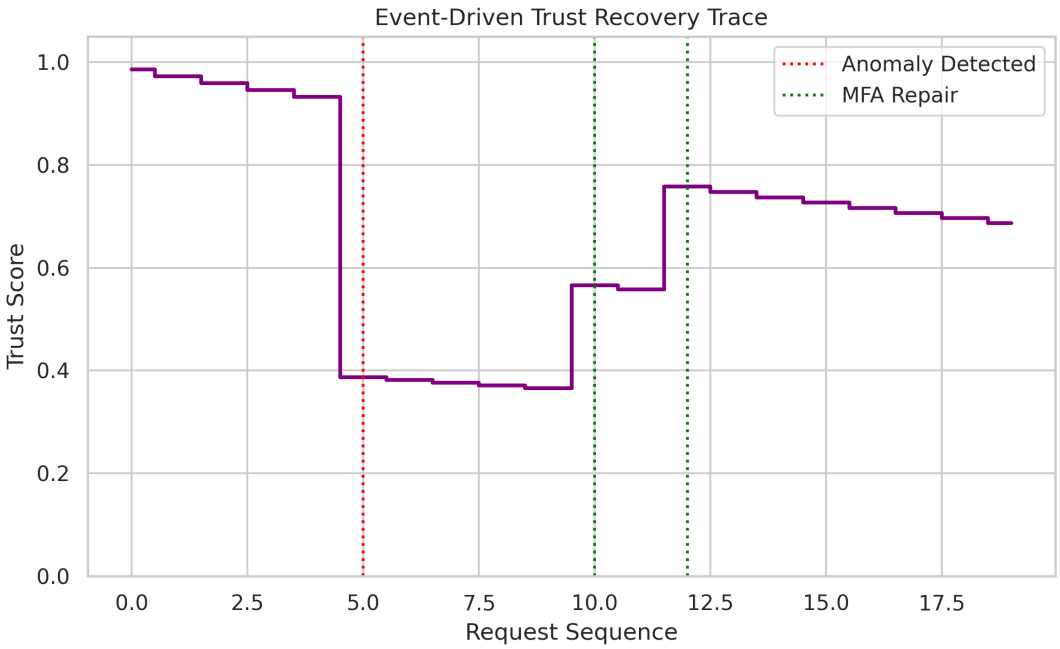

Fig. 12: Trace visualization of continuous trust decay over time, followed by a sharp recovery spike resulting from a successful MFA verification.

nuanced threats. Rather than statically waiting for an arbitrary 60-minute token expiration event while an attacker quietly harvests data, the QSRAC system degrades its trust in real-time, autonomously stepping up security protocols before catastrophic thresholds are breached.

Despite its theoretical and empirical robustness, the architecture inherently exhibits limitations tied directly to the fundamental nature of network telemetry data. The extreme, unyielding polarization of the UNSW-NB15 dataset (where transactions are essentially binary: explicitly safe administrative tasks or explicitly hostile exploits) resulted in exceptionally narrow intermediate risk bands, even after logit calibration. Consequently, the Machine Learning model exhibits high non-linear sensitivity; minimal changes in the `request_rate` parameter can abruptly push a session directly from 'Low' to 'High' risk, bypassing the 'Medium' buffer zone. While mathematically accurate and protective, this high sensitivity can cause operational friction in volatile, high-velocity enterprise environments, leading to unpredictable MFA interruptions for highly active employees.

Additionally, while deploying Redis as the arbiter of state provides the necessary microsecond atomicity, it inherently introduces systemic fragility. If the Redis cluster is not sharded, replicated, and geographically distributed correctly, it becomes a single, catastrophic point of failure that could render the entire API gateway completely inoperable during a localized outage.

XVII. Practical Implications and Deployment Considerations

While the QSRAC framework demonstrates strong theoretical rigor and empirical performance, its practical value must be evaluated within real-world enterprise environments characterized by strict latency constraints, heterogeneous infrastructure, and evolving threat landscapes. Translating the proposed architecture into production systems therefore requires careful consideration of scalability, deployment feasibility, and operational stability.

A key strength of QSRAC lies in its modular architectural design. Each layer—ranging from quantum-safe session initialization to Redis-based atomic validation—operates independently while maintaining overall system coherence. This modularity enables incremental adoption within existing Zero Trust deployments. In practice, organizations may initially integrate the hybrid machine learning risk engine alongside

conventional access control systems, utilizing calibrated risk outputs without immediately transitioning to fully stateful enforcement. Additional components, such as trust decay modeling and hash-chain validation, can be incorporated progressively with minimal disruption.

From a performance perspective, the system achieves a balanced trade-off between enhanced security guarantees and operational efficiency. As demonstrated in Section XI, the mean end-to-end inference latency is 7.91 ms, satisfying the sub-10 ms requirement typical of high-throughput microservice environments. This efficiency is achieved by restricting Post-Quantum Cryptographic (PQC) operations to the initial session establishment phase, while continuous request validation is handled using lightweight symmetric primitives such as HMAC-SHA256 [28], [32].

Another important implication is the system's resilience against evolving threat strategies. Traditional access control mechanisms often rely on static thresholds that degrade over time due to concept drift. In contrast, QSRAC introduces adaptability across multiple layers. The hybrid machine learning engine enables detection of both known attack patterns and previously unseen anomalies [30], [33]. Additionally, calibrated probability outputs improve stability under highly skewed cybersecurity data distributions.

The continuous trust decay model further strengthens system responsiveness by incorporating temporal risk accumulation. Instead of maintaining binary trust states, QSRAC models trust as a continuous variable that evolves based on contextual risk and environmental sensitivity. This enables effective mitigation of low-and-slow attack strategies while enforcing step-up authentication only when necessary [10], [34].

Despite these advantages, several deployment considerations must be addressed. The Redis-based state management layer, while highly efficient, introduces operational and scalability considerations. Production deployments require careful configuration of clustering, replication, and failover strategies to ensure reliability and avoid single points of failure [12].

Similarly, the adoption of Post-Quantum Cryptography introduces increased key sizes and bandwidth overhead compared to classical approaches [8], [15], [16]. As a result, hybrid cryptographic strategies may be necessary during transitional phases to maintain compatibility with existing systems.

Finally, user experience must be considered in continuous authentication environments. Excessive multi-factor authentication (MFA) prompts may lead to user fatigue. QSRAC mitigates this by employing gradual trust degradation, ensuring that additional verification is triggered only when justified by elevated risk levels [25].

Overall, QSRAC demonstrates that it is feasible to integrate stateful security enforcement, continuous trust modeling, and quantum-resistant cryptographic mechanisms into a unified Zero Trust framework while maintaining practical performance requirements.

XVIII. Future Work

Future iterations of this research will focus heavily on transitioning the centralized Redis state-machine into a decentralized, distributed ledger environment (e.g., employing highly optimized Byzantine Fault Tolerant consensus algorithms) to enhance geographical fault tolerance and eliminate single points of failure entirely.

Additionally, we intend to deeply explore continuous federated learning paradigms. This would allow the Hybrid ML engine to ingest, train upon, and adapt to non-polarized, enterprise-specific telemetry data continuously at the edge, without requiring centralized model retraining, thereby fully preserving corporate data privacy. Finally, we will investigate the integration of advanced biometric telemetry streams—such as keystroke dynamics and localized mouse behavioral telemetry—directly into the contextual extraction layer to provide higher fidelity, less volatile inputs for the trust decay equations, further smoothing the user experience.

XIX. Conclusion

The imminent, inevitable paradigm shift toward cryptographically relevant quantum computing mandates an immediate and fundamental redesign of enterprise access control architectures. This paper presented the

Quantum-Safe Risk-Adaptive Access Control (QSRAC) framework, successfully merging Post-Quantum Cryptography standards, Hybrid Ensemble Machine Learning, and rigorous Stateful Hash-Chain integrity into a highly cohesive, high-performance Zero Trust API gateway.

Our extensive theoretical analysis and empirical evaluations conclusively demonstrate that QSRAC achieves highly accurate risk identification (F1-score 0.9071), processes stateful cryptographic transactions with ultra-low latency (mean 7.91 ms), and mathematically invalidates replay, state-tampering, and session hijacking attacks via atomic Lua-Redis operations. The introduction of explicit logit-calibrated risk thresholds, operating alongside a differential mathematical trust decay engine, ensures that access is dynamically and fluidly modulated based on real-time behavioral context, effectively enforcing the core Zero Trust tenet of "never trust, always verify" at an unbreakable, cryptographic level.

REFERENCES

[1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, Aug. 2020.

[2] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, IEEE, 1994, pp. 124–134.

[3] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.

[4] K. Al-Tawil, "Intrusion detection using LightGBM and XGBoost," *IEEE Access*, vol. 8, pp. 43543–43555, 2020.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*, IEEE, 2008, pp. 413–422.

[6] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to attribute based access control (ABAC) definition and considerations," *NIST Special Publication 800-162*, 2014.

[7] M. A. Alshehri, et al., "A continuous trust evaluation model for Zero Trust architectures," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1253–1268, 2021.

[8] NIST, "Post-Quantum Cryptography Standardization," *National Institute of Standards and Technology*, 2024. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography

[9] A. Ali, et al., "Integrating Post-Quantum Cryptography and Blockchain to Secure Low-Cost IoT Devices," *IEEE Xplore*, 2024.

[10] A. L. Barabási, "Risk-Adaptive Access Control (RAdAC) in dynamic network environments," *Journal of Network Security*, vol. 12, pp. 45–56, 2018.

[11] M. Campagna, et al., "Quantum-Safe Cryptography and Security," *ETSI White Paper No. 8*, 2015.

[12] J. Carlson, "Redis in Action," *Manning Publications Co.*, 2013.

[13] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Advances in large margin classifiers*, vol. 10, no. 3, pp. 61–74, 1999.

[14] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in *2015 military communications and information systems conference (MilCIS)*, IEEE, 2015, pp. 1–6.

[15] R. Avanzi, et al., "CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation," *NIST PQC Round 3 Submission*, 2020.

[16] L. Ducas, et al., "CRYSTALS-Dilithium Algorithm Specifications and Supporting Documentation," *NIST PQC Round 3 Submission*, 2020.

[17] S. Ramírez, "Building highly performant APIs with FastAPI," *Python Developers Conference*, 2021.

[18] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[19] L. Lamport, "Password authentication with insecure communication," *Communications of the ACM*, vol. 24, no. 11, pp. 770–772, 1981.

[20] C. E. Landwehr, A. R. Bull, J. P. McDermott, and W. S. Choi, "A taxonomy of computer program security flaws," *ACM Computing Surveys (CSUR)*, vol. 26, no. 3, pp. 211–254, 1994.

[21] D. Stebila and M. Mosca, "Post-quantum TLS without handshake signatures," in *International Conference on Cryptology and Information Security in Latin America*, Springer, 2020.

[22] N. Papernot, et al., "The limitations of deep learning in adversarial settings," in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016.

[23] A. Ghorbani, "Network Intrusion Detection and Prevention: Concepts and Techniques," *Springer Science & Business Media*, 2009.

[24] D. R. Kuhn, E. J. Coyne, and T. R. Weil, "Adding attributes to role-based access control," *IEEE Computer*, vol. 43, no. 6, pp. 79–81, 2010.

[25] A. P. A. F. da Silva, et al., "Continuous authentication: A systematic literature review," *Computer Networks*, vol. 182, 107474, 2020.

[26] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *RFC 7519*, IETF, 2015.

[27] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?" *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[28] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," *RFC 2104*, IETF, 1997.

[29] J. Bogner, et al., "Assessing the impact of microservices on the performance of a software architecture," *Proceedings of the 2018 International Conference on Software Architecture*, 2018.

[30] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.

[31] P. R. Kumar, et al., "AI-Driven Post-Quantum Cryptography for Cyber-Resilient V2X Communication in Transportation Cyber-Physical Systems," arXiv:2510.08496v1, 2026.

[32] S. M. Raza, et al., "Performance Evaluation of Post-Quantum Cryptography: A Comprehensive Framework for Experimental Analysis," *Journal of Information Systems Engineering and Management*, 2026.
[33] Y. Xin, L. Kong, Z. Liu, et al., "Machine learning and deep learning methods for cybersecurity," *IEEE Access*, vol. 6, pp. 35365–35381, 2018.
[34] S. Syed, et al., "A comprehensive survey on zero trust network architecture," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 1, pp. 636–666, 2022.
[35] A. K. Ekert, "Quantum cryptography based on Bell's theorem," *Physical Review Letters*, vol. 67, no. 6, pp. 661, 1991.